

BACon: Efficient Batch Processing of Counting Queries

Yuxi Liu
Duke University
yuxi.liu@duke.edu

Pankaj K. Agarwal
Duke University
pankaj@cs.duke.edu

Xiao Hu
University of Waterloo
xiaohu@uwaterloo.ca

Jun Yang
Duke University
junyang@cs.duke.edu

ABSTRACT

Counting queries are ubiquitous in database systems, particularly for driving internal system optimization. Learned models for cardinality estimation rely heavily on large-scale training data, yet generating such data by executing massive batches of counting queries is expensive. We propose BACon, an efficient algorithm for batch evaluation of counting queries on top of a database system, without modifying its internals. BACon integrates the idea of factorized databases with a workload-aware domain quantization strategy, allowing it to evaluate batches of counting queries using compact data structures rather than materializing massive join results. BACon’s design is compatible with most database management system, and we have implemented it as a client-side application on PostgreSQL with a lightweight C-language UDF (user-defined function). This implementation delivers speedups between 2× and 178× over baselines and good performance across various workloads, making training and maintenance of learned cardinality estimation models significantly more practical.

PVLDB Reference Format:

Yuxi Liu, Xiao Hu, Pankaj K. Agarwal, and Jun Yang. BACon: Efficient Batch Processing of Counting Queries. PVLDB, 14(1): XXX-XXX, 2020. doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/louisja1/bacon>.

1 INTRODUCTION

Batches of counting queries are not only useful in their own right for database applications, but also frequently serve to collect basic statistics from data for monitoring and optimization. With the growing popularity of *learned query optimization* [12, 46, 47, 82] in recent years, an interesting workload has emerged: collecting training data for *learned cardinality estimation* (CE) [13, 27, 29, 40, 50, 51, 58, 60, 71]. CE is a critical component of query optimization, as its accuracy directly impacts the quality of query execution plans [35, 61]. In these workloads, queries typically involve joins and selections over base tables but report only the final counts of the result sets. These query-count pairs are subsequently used to train

CE models.¹ Such queries may be derived from past workloads or synthesized specifically for training. While the number of distinct join patterns is naturally limited by the database schema, the queries themselves additionally contain a variety of selection conditions, targeting different attributes with varying constants. Beyond simple equality comparisons, many conditions involve inequality or range predicates. Table 1 summarizes nine public query workloads widely used in CE research for training and evaluation. These workloads span three commonly studied benchmark databases—IMDB [35], STATS [18] and DSB [10]—and capture a broad range of query shapes, join structures, and predicate types.

Executing such workloads is time-consuming. Even when viewed as a one-time training cost, the overhead can be daunting; in our experience, some workloads require hours or even more than a day to execute (Section 5). Furthermore, models are rarely trained once and then forgotten. As the database state changes, they can become outdated [32, 37]: cardinalities from previously executed queries may no longer be accurate, necessitating the re-execution of queries to retrain the models. While monitoring real result counts from query execution feedback can mitigate this issue, simply observing final query result counts is insufficient [18, 29, 60]. Effective training requires (1) the size of intermediate results for subqueries within *potentially optimal* plans, not just the plan actually executed, and (2) feedback from unseen queries to prepare for potential workload shifts [37, 72, 75]. In a continuously running environment, executing these monitoring or training queries must not disrupt normal workloads. These observations underscore the need for more efficient support for executing batches of counting queries.

Given the extensive literature on query processing, many ideas are applicable to the general problem of batch counting queries—including multi-query optimization, scalable continuous query processing, and factorized databases—see Section 6 for more discussion. A reasonable starting point, leveraging multi-query optimization, is to exploit the fact that many queries share the same join pattern, differing only in their selection conditions. Instead of executing them independently, one could perform the full join first and then apply individual selection conditions to obtain per-query counts. While this baseline enables shared join processing, it leads to materializing the full join result, which can be massive for large databases.

We propose **BACon**, a practical, scalable algorithm for efficient *Batch processing of Counting queries*. A key idea, inspired by work on factorized databases [7, 8, 56], is that a counting query over a join can be evaluated without enumerating the join result. While the count operator cannot generally be pushed down through a join, if

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

¹More discussions on how CE models leverage query-count pairs are in Section 6.

Table 1: Summary of nine query workloads across three databases, including the workload name with the link to file, associated database, number of queries (Qs), number of tables per query (Ts), number of join patterns (formally defined in Section 2), and a short description with source in the second row for each workload.

Query Workload	DB	# Qs	# Ts	# join patterns
synthetic [31]	IMDB	5,000	1 to 3	16
Kipf et al. [29]: generated by the training data generator with a different seed				
scale [30]	IMDB	500	1 to 5	31
Kipf et al. [29]: designed to show how MSCN generalizes to more joins.				
job-light [28]	IMDB	70	2 to 5	18
Kipf et al. [29]: derived from JOB [35], excluding string predicates and disjunctions.				
job-light-single [20]	IMDB	254	1	1
Han et al. [18]: single-table sub-plan queries extracted from job-light.				
job-light-join [19]	IMDB	696	2 to 5	27
Han et al. [18]: join sub-plan queries extracted from job-light.				
stats-ceb [24]	STATS	146	2 to 7	58
Han et al. [18]: designed to be more comprehensive, with more diverse queries and more complex join patterns on STATS, compared to job-light on IMDB.				
stats-ceb-single [23]	STATS	632	1	1
Han et al. [18]: single-table sub-plan queries extracted from stats-ceb.				
stats-ceb-join [22]	STATS	2,603	2 to 7	120
Han et al. [18]: join sub-plan queries extracted from stats-ceb.				
dsb-grasp-20k [41]	DSB	20,000	1 to 5	16
A sample of the original workload [74] generated by SeConCDF [75] and GRASP [73] on DSB. From 149,828 original queries, we remove queries not applicable to our methods and randomly sample 20,000 queries (details in Section 5).				

the join is processed in a “factorized” manner—grouped by joining attribute values—we can simply count the joining “factors” and multiply these counts, avoiding enumeration of the joined result tuples. A second key idea is workload-aware quantization of selection attribute domains. This idea allows the data required for counting queries to be compressed, naturally exploiting the overlap and sharing of selection predicates within the workload. BACON seamlessly combines these ideas, using compact “count maps” rather than bloated join tuples to represent intermediate results; to combine these intermediate results, it uses a pair of operators $\otimes$ and $\oplus$, defined with clear semantics and amenable to optimization. Finally, to ensure practicality and ease of adoption, we implement BACON as a client application on top of a database system (DBMS) without modifying its internals. BACON leverages the DBMS for processing, utilizes user-defined functions (UDFs), and carefully balances database- and client-side execution to minimize interface overhead.

Our experimental results show that BACON is highly competitive for training workloads for learned CE in Table 1, achieving 2× to 178× speedups over baselines. Moreover, BACON performs well across diverse workloads, providing significant speedups for expensive join patterns while remaining competitive on those where baseline approaches are already efficient.

2 PRELIMINARIES

Problem Statement and Notations. Consider a database $\mathcal{D}$ with n tables $R_1, \dots, R_n$, where each table R_i has a set of attributes denoted $\text{Attrs}[R_i]$. We are interested in processing a set of counting queries Q over $\mathcal{D}$.² Each query counts the number of tuples returned by a

Listing 1: First 4 queries in stats_ceb [24].

```

SELECT COUNT(*) FROM badges as b, users as u -- Q1
WHERE b.UserId = u.Id AND u.UpVotes >= 0;
SELECT COUNT(*) FROM comments as c, badges as b -- Q2
WHERE c.UserId = b.UserId AND c.Score = 0 AND b.Date <=
'2014-09-11 14:33:06'::timestamp;
SELECT COUNT(*) FROM comments as c, postHistory as ph -- Q3
WHERE c.UserId = ph.UserId AND c.Score = 0 AND
ph.PostHistoryTypeId = 1;
SELECT COUNT(*) FROM comments as c, postHistory as ph -- Q4
WHERE c.UserId = ph.UserId AND ph.PostHistoryTypeId = 1 AND
ph.CreationDate >= '2010-09-14 11:59:07'::timestamp;

```

single-table selection or a selection-join over a subset of the tables in $\mathcal{D}$. We assume equality joins with no cycles or self-joins, and that the selection is a conjunction of predicates comparing a single attribute with a literal using $=$, $>$, $<=$, etc. We begin by introducing some useful notations (a table of notations is presented in the full version [42]). Given each query $Q \in \mathcal{Q}$:

- Tables_Q denotes the subset of tables in $\mathcal{D}$ referenced by Q .
- $\text{Attrs}_Q^\bowtie$ denotes the subset of attributes in Tables_Q referenced by the join predicates of Q .
- $\text{Preds}_Q^\bowtie$ denotes Q ’s join predicates, represented as the set of equivalent classes of attributes in $\text{Attrs}_Q^\bowtie$ induced by Q ’s join predicates. Two attributes A_1, A_2 belong to the same equivalent classes iff $A_1 = A_2$ is logically implied by Q ’s join predicates.
- Given a pair of disjoint subsets $\mathcal{R}$ and $\mathcal{R}'$ of tables in Tables_Q , $\text{Attrs}_Q^\bowtie[\mathcal{R}|\mathcal{R}']$ denotes the subset of attributes of $\mathcal{R}$ used by Q to join with $\mathcal{R}'$; i.e., $\text{Attrs}_Q^\bowtie[\mathcal{R}|\mathcal{R}'] = \{A \in \bigcup_{R \in \mathcal{R}} \text{Attrs}[R] \mid \exists \mathcal{E} \in \text{Preds}_Q^\bowtie : A \in \mathcal{E} \wedge (\exists A' \in \mathcal{E} : A' \in \bigcup_{R \in \mathcal{R}'} \text{Attrs}[R])\}$.
- Attrs_Q^σ denotes the subset of attributes in Tables_Q referenced by Q ’s selection predicates.
- Preds_Q^σ denotes Q ’s selection predicates, represented as a mapping from each attribute $A \in \text{Attrs}_Q^\sigma$ to a range G over the domain of A , such that G is the widest range such that $A \in G$ is logically implied by Q ’s selection predicates.
- Given a subsets $\mathcal{R}$ of tables in Tables_Q , $\text{Attrs}_Q^\sigma[\mathcal{R}]$ denotes the set of attributes of $\mathcal{R}$ referenced by Q ’s selection conditions; i.e., $\text{Attrs}_Q^\sigma[\mathcal{R}] = \text{Attrs}_Q^\sigma \cap \bigcup_{R \in \mathcal{R}} \text{Attrs}[R]$.

For example, Listing 1 shows the first four queries in the workload stats_ceb [24] over the STATS database [18]. For Q_4 :

- $\text{Tables}_{Q_4} = \{\text{comments}, \text{postHistory}\}$;
- $\text{Attrs}_{Q_4}^\bowtie = \{\text{comments.UserId}, \text{postHistory.UserId}\}$;
- $\text{Preds}_{Q_4}^\bowtie = \{\{\text{comments.UserId}, \text{postHistory.UserId}\}\}$ (both join attributes are in one equivalence class);
- $\text{Attrs}_{Q_4}^\bowtie[\{\text{comments}\}|\{\text{postHistory}\}] = \{\text{comments.UserId}\}$;
- $\text{Attrs}_{Q_4}^\bowtie[\{\text{postHistory}\}|\{\text{comments}\}] = \{\text{postHistory.UserId}\}$;
- $\text{Attrs}_{Q_4}^\sigma = \{\text{postHistory.PostHistoryTypeId}, \text{postHistory.CreationDate}\}$;
- $\text{Preds}_{Q_4}^\sigma = \{\text{postHistory.PostHistoryTypeId} \mapsto [1, 1], \text{postHistory.CreationDate} \mapsto ['2010-09-14 11:59:07'::\text{timestamp}, \infty)\}$.

The First Baseline: Independent Processing (INDPROC). A straightforward solution, adopted by most existing work on learned CE,

²For simplicity, we discuss only counting queries here, but our methods easily generalize to all *distributive* and *algebraic* SQL aggregates [17] including also SUM, MIN, MAX,

AVG, and STDDEV, where each input expression is a single attribute or computable over attributes from the same table.

runs the queries in $\mathcal{Q}$ one by one, independently, using the DBMS that manages $\mathcal{D}$. We call this approach **INDPROC** for short. The performance of **INDPROC** is heavily dependent on the underlying DBMS. A capable DBMS will optimize each query by pushing down selection conditions, reordering joins, and choosing appropriate join methods, leveraging data statistics and available indices. Caching by the DBMS buffer pool may also improve execution performance across queries. On the other hand, most DBMS lack advanced methods for optimizing multiple queries simultaneously. Moreover, most of them choose to execute a counting query Q by first joining all tables in Q before applying the final COUNT aggregation, failing to explore opportunities for pushing COUNT below joins.

Join Patterns. Before presenting a more advanced baseline solution as well as our solution in Section 3, we introduce a concept used by both. The *join pattern* of a query Q is characterized by $\langle \text{Tables}_Q, \text{Preds}_Q^\sigma \rangle$, i.e., the tables that Q joins, and Q 's join predicates. Suppose queries in $\mathcal{Q}$ have q distinct join patterns $\mathfrak{J}_1, \dots, \mathfrak{J}_q$; these patterns partition $\mathcal{Q}$ into a disjoint union of q subsets of queries, denoted $\mathcal{Q}[\mathfrak{J}_1], \dots, \mathcal{Q}[\mathfrak{J}_q]$, where each $\mathcal{Q}[\mathfrak{J}_i]$ is the set of queries with join pattern $\mathfrak{J}_i$ (but may differ in their selection predicates). For example, Q_3 and Q_4 in Listing 1 have the same join pattern (with tables {comments, postHistory} and join predicate comments.UserId = postHistory.UserId, despite having different selection predicates). Q_1 and Q_2 each contribute a distinct join pattern. In practice, because all queries in $\mathcal{Q}$ come from the same underlying database $\mathcal{D}$, the number of distinct join patterns tends to be much smaller than the number of queries. For example, JOB's synthetic [31] contains 5,000 queries but only 16 distinct join patterns.

The Second Baseline: Join and Post-Filtering (PostFILT). This approach seeks to avoid redundant computation across queries with identical join patterns. We pre-process the queries into partitions $\mathcal{Q}[\mathfrak{J}_1], \dots, \mathcal{Q}[\mathfrak{J}_q]$ according to their join patterns. This step requires only a scan over $\mathcal{Q}$ to perform syntactical analysis. For each partition $\mathcal{Q}[\mathfrak{J}_i]$, we compute the join only once, followed by a post-filtering step, which checks each join result tuple against the selection predicates of queries in $\mathcal{Q}[\mathfrak{J}_i]$ to determine which queries' counts to increment. We call this baseline **PostFILT**.

There are many options for implementing the post-filtering step, as discussed in Section 6, including methods that first build a data structure for $\mathcal{Q}[\mathfrak{J}_i]$ for efficient identification and updating of counters in time sublinear to $|\mathcal{Q}[\mathfrak{J}_i]|$ per join result tuple. While these methods can scale to thousands or millions of queries, the typical workloads we target do not have so many queries per join pattern, as evidenced in Table 1. Through our experiments, we have found a simple strategy leveraging the underlying DBMS to be the most effective. Given a join pattern $\mathfrak{J} = \langle \text{Tables}_{\mathfrak{J}}, \text{Preds}_{\mathfrak{J}}^\sigma \rangle$ and k queries $Q_1, \dots, Q_k$ sharing this pattern, we issue a single SQL query in Listing 2 to compute all of them. Here, Preds^σ , Tables , and Preds^σ are translated into SQL. Each CASE expression determines whether a join result tuple contributes to a query Q_i by checking its selection predicates (a failed check yields NULL, which is ignored by COUNT). In the end, a single k -component result tuple is computed, with each component holding the result count for one query.

Discussion. Compared with **INDPROC**, **PostFILT** eliminates redundant computation of joins among queries sharing the same join

Listing 2: Computing k queries with join pattern $\mathfrak{J}$ in **PostFILT**.

```
SELECT COUNT(CASE WHEN Preds_{Q_1}^\sigma THEN 1 END), ...,
COUNT(CASE WHEN Preds_{Q_k}^\sigma THEN 1 END)
FROM Tables_{\mathfrak{J}} WHERE Preds_{\mathfrak{J}}^\sigma;
```

pattern. However, **INDPROC** can still outperform **PostFILT** if the selection predicates of these queries have little overlap, and if database indices enable **INDPROC** to apply selective selection predicates early in query processing. In contrast, **PostFILT** has no effective means to push selection predicates down because in most workloads, the disjunction of all selection predicates from multiple queries cannot be expressed as a succinct, “sargable” [64] WHERE condition without introducing many false positives. Furthermore, most database optimizers do not push aggregation below joins, let alone those with conditional expression inputs as in Listing 2. Therefore, **PostFILT** effectively enumerates the full join result before post-filtering and counting—a key limitation that we seek to overcome.

3 BASIC BACON

This section introduces basic **BACON**, focusing on key ideas and the high-level algorithm. Many implementation and optimization details are also crucial to making **BACON** competitive in practice, but to simplify presentation, we defer them to Section 4. We start with three key ideas, along with some results and definitions based on them; we then describe the algorithm. Like **PostFILT**, given a set $\mathcal{Q}$ of queries, **BACON** partitions $\mathcal{Q}$ into subsets according to join patterns, such that queries in each subset $\mathcal{Q}[\mathfrak{J}]$ share the same join pattern $\mathfrak{J} = \langle \text{Tables}_{\mathfrak{J}}, \text{Preds}_{\mathfrak{J}}^\sigma \rangle$. Hence, most of this section focuses on how to process one such subset of queries given $\mathfrak{J}$.

Before proceeding, we briefly present a geometric view of the problem for intuition. Each result tuple in the cross product of $\text{Tables}_{\mathfrak{J}}$ can be seen as a point in a high-dimensional space $\mathbb{S}$, where each dimension corresponds to an attribute in $\bigcup_{R \in \text{Tables}_{\mathfrak{J}}} \text{Attrs}[R]$, ignoring those not referenced by any predicate in $\mathcal{Q}[\mathfrak{J}]$. Let J denote the result of the full join of $\text{Tables}_{\mathfrak{J}}$ using $\text{Preds}_{\mathfrak{J}}^\sigma$. Points in J are those cross-product points that lie on a hyperplane $\mathbb{J}$ in $\mathbb{S}$ defined by $\text{Preds}_{\mathfrak{J}}^\sigma$. Each query $Q \in \mathcal{Q}[\mathfrak{J}]$ corresponds to an orthogonal hyperrectangle in $\mathbb{S}$ with range predicates restricted to dimensions in Attrs_Q^σ . Our problem boils down to counting, for each Q , how many points in J fall into Q 's hyperrectangle.

Intuitively, these points are not positioned arbitrarily: not only do they lie on $\mathbb{J}$ because of the join predicates, but they also come from the cross product of $\text{Tables}_{\mathfrak{J}}$, meaning that their projections onto the subspace $\mathbb{S}[\text{Attrs}[R]]$ for each $R \in \text{Tables}_{\mathfrak{J}}$ cannot exceed $|R|$ distinct points, which is often much smaller than $|J|$. In fact, if we consider the subset of J restricted to any particular combination of values for all join attributes, their projections onto $\mathbb{S}[\bigcup_{R \in \text{Tables}_{\mathfrak{J}}} \text{Attrs}[R]]$ will form a perfect Cartesian product of their projections onto each of $\mathbb{S}[\text{Attrs}[R]]$ for $R \in \text{Tables}_{\mathfrak{J}}$ (which we shall substantiate in Section 3.1). This property, along with the fact the query hyperrectangles may overlap significantly, makes it possible to perform counting tasks more efficiently than simply enumerating J upfront (which **PostFILT** does).

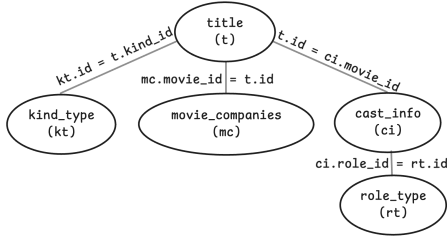


Figure 1: A join pattern in the IMDB schema [36].

To further exploit the overlaps among the query hyperrectangles, consider a subset of J that forms a Cartesian product described above. We can partition its projection onto each subspace $\mathbb{S}[\text{Attrs}[R]]$ into a coarse grid, using the endpoints of the query range predicates (a process we call “quantization” later in Section 3.2). For example, in Figure 2, this projection consists of a collection of red points, partitioned by a 4×4 grid. A key property of this grid is that all points within a cell lie in the same subset of query hyperrectangles. Therefore, we can compress the red points in each cell to a single “weighted” point, yielding a compressed representation of J (which we refer to as a “count map” later in Section 3.3). This compressed representation is constructed recursively for each $\mathfrak{J}$, without having to enumerate J first (Section 3.4).

3.1 Conditional Orthogonality of Joins

The first idea has its roots in the well-studied problem of factorized databases and related worst-case optimal join algorithms (further discussed in Section 6). A simple observation is that, in order to count the number of tuples in a cross product between two tables R_1 and R_2 , we just need to calculate $|R_1| \times |R_2|$, without enumerating $R_1 \times R_2$. While the same observation no longer holds for $|R_1 \bowtie_{R_1.A=R_2.A} R_2|$ when join predicate exists, if we additionally set the join attribute value $A = x$, then the number of result tuples *conditioned on this specific setting* can still be computed directly: i.e., $|\sigma_{A=x}(R_1 \bowtie_{R_1.A=R_2.A} R_2)| = |\sigma_{A=x} R_1| \times |\sigma_{A=x} R_2|$. We can generalize this idea further to a star-shaped join as follows.

LEMMA 3.1 (CONDITIONAL ORTHOGONALITY OF JOINS). *Consider a star-shaped join query centered at E_0 :*

$$E_0 \bowtie_{\theta_1} E_1 \bowtie_{\theta_2} \cdots \bowtie_{\theta_n} E_n.$$

Here, the E_i ’s are subqueries, and for each $i = 1, \dots, n$, θ_i is a conjunctive predicate equating pairs of attributes from E_0 and E_i . Note that there are no join predicates across $E_1, \dots, E_n$. Denote by $\mathcal{A}_0$ the set of join attributes from E_0 referenced by $\theta_1, \dots, \theta_n$. Let v be any mapping of every attribute $A \in \mathcal{A}_0$ to a value $v(A)$ in A ’s domain, and let $v[\![\theta_i]\!]$ denote the condition obtained by applying v to θ_i (i.e., replacing each $A \in \mathcal{A}_0$ by $v(A)$ —note that the resulting condition becomes a selection over E_i). The following equivalence holds:

$$\begin{aligned} & \sigma_{\bigwedge_{A \in \mathcal{A}_0} A=v(A)} (E_0 \bowtie_{\theta_1} E_1 \bowtie_{\theta_2} \cdots \bowtie_{\theta_n} E_n) \\ &= \left(\sigma_{\bigwedge_{A \in \mathcal{A}_0} A=v(A)} E_0 \right) \times \left(\sigma_{v[\![\theta_1]\!]} E_1 \right) \times \cdots \times \left(\sigma_{v[\![\theta_n]\!]} E_n \right). \end{aligned}$$

EXAMPLE 3.1. Consider the join pattern in Figure 1. Tables title AS t , kind_type AS kt , and movie_companies AS mc correspond to E_0, E_1, E_2 , while the join between cast_info AS ci and role_type AS rt corresponds

to E_3 . In title, suppose we fix $t.\text{kind_id}=x$ (which joins $kt.\text{id}$) and $t.\text{id}=y$ (which joins $mc.\text{movie_id}$ and $ci \bowtie rt$). By Lemma 3.1, the number of full join result tuples with $(t.\text{kind_id}, t.\text{id}) = (x, y)$ can be computed directly as: $|\sigma_{t.\text{kind_id}=x \wedge t.\text{id}=y} t| \times |\sigma_{kt.\text{id}=x} kt| \times |\sigma_{mc.\text{movie_id}=y} mc| \times |\sigma_{ci.\text{movie_id}=y} (ci \bowtie rt)|$, without enumerating the full join result tuples. To compute the full join result size overall, we iterate over all possible combinations of $(t.\text{kind_id}, t.\text{id})$ values in t , apply the above procedure to each combination, and tally the total.

The style of processing illustrated by the above example has been used recently for efficient computation of aggregate queries [33]. The subexpression $|\sigma_{ci.\text{movie_id}=y} (ci \bowtie rt)|$ can be processed by the same procedure. Later in this section, we will see how to extend the idea to computing result counts of multiple queries with different selection predicates beyond simply counting the full join result.

3.2 Quantization of Selection Attributes

While tables and attribute domains can be large, the number of queries in $\mathcal{Q}$ places a natural constraint on the number of constants from each domain appearing in selection predicates. In other words, from the perspective of $\mathcal{Q}$, fine-grained differences among attribute values may not affect result counts. Our key idea is to compress the attribute domains using workload-aware *quantization*, turning large, complex domains into a small range of integers that are efficient to work with.

Given a join pattern $\mathfrak{J}$, let $\text{Attrs}_{\mathfrak{J}}^{\sigma} = \bigcup_{Q \in \mathcal{Q}[\mathfrak{J}]} \text{Attrs}_Q^{\sigma}$ denote the set of selection attributes in all queries of pattern $\mathfrak{J}$. We construct a *quantization scale* $\mathfrak{b}_A$ for each selection attribute $A \in \text{Attrs}_{\mathfrak{J}}^{\sigma}$ as follows. First, we extract from $\mathcal{Q}[\mathfrak{J}]$ the set $\text{Preds}_{\mathfrak{J}}^{\sigma}(A) = \{\text{Pred}_Q^{\sigma}(A) \mid \exists Q \in \mathcal{Q}[\mathfrak{J}] : A \in \text{Pred}_Q^{\sigma}\}$ of predicate ranges associated with A . We sort all range endpoints, which partition the domain of A into an ordered list of atomic ranges. For each atomic range contained in at least one query range in $\text{Preds}_{\mathfrak{J}}^{\sigma}(A)$, we create a new *bucket* in $\mathfrak{b}_A$ and assign it a serial integer id (starting with 1). Hence, $\mathfrak{b}_A$ maps each relevant atomic range (bucket) to an integer, preserving order. A value x from A ’s domain is quantized into an integer $\mathfrak{b}_A(x)$, the id of the bucket containing x , or 0 if x lies outside all of buckets in $\mathfrak{b}_A$. We present the detailed construction algorithm in the full version [42], which handles additional intricacies with open or close intervals.

Let $\mathfrak{B}_{\mathfrak{J}} = \{\mathfrak{b}_A \mid A \in \text{Attrs}_{\mathfrak{J}}^{\sigma}\}$ denote the collection of all attribute quantization scales for queries in $\mathcal{Q}[\mathfrak{J}]$, and $\mathfrak{B}_{\mathfrak{J}}[R] = \{\mathfrak{b}_A \mid A \in \text{Attrs}_{\mathfrak{J}}^{\sigma} \cap \text{Attrs}[R]\}$ denote those for attributes in table $R \in \text{Tables}_{\mathfrak{J}}$. Returning to the geometric view introduced at the beginning of the section, $\mathfrak{B}_{\mathfrak{J}}[R]$ induces a grid over the subspace of $\mathbb{S}$ spanning the selection attribute dimensions of R . This grid allows us to map each tuple in R , by its selection attribute values $\langle x_1, \dots, x_k \rangle$, to a *grid coordinate* $\mathfrak{b} = (b_1, \dots, b_k)$, where each $b_i = \mathfrak{b}_{A_i}(x_i)$ is the bucket id for value x_i in the quantization scale for attribute A_i .

Continuing with the geometric intuition, collectively, quantization scales $\mathfrak{B}_{\mathfrak{J}}$ induces a grid over the subspace $\mathbb{S}[\text{Attrs}_{\mathfrak{J}}^{\sigma}]$ consisting of the selection attribute dimensions across all tables in $\text{Tables}_{\mathfrak{J}}$. By construction of $\mathfrak{B}_{\mathfrak{J}}$, all query hyperrectangles in $\mathcal{Q}[\mathfrak{J}]$ perfectly align with grid boundaries. Hence, all selection predicates can be quantized using the same scales. Lemma 3.2 below formalizes the

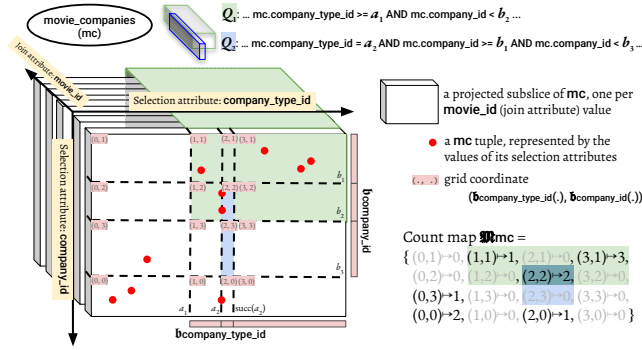


Figure 2: Quantized selection count map for a projected subslice of table *mc*. The quantization scales for *mc* are induced by queries Q_1 and Q_2 (only selection predicates on *mc* are shown).

guarantee that precise evaluation of selection predicates is possible in the quantized space.

LEMMA 3.2 (QUANTIZATION PRESERVES SELECTIONS). *Given a set of selection-join queries $\mathcal{Q}[\mathfrak{J}]$ and quantization scales $\mathfrak{B}_{\mathfrak{J}}$ constructed from $\mathcal{Q}[\mathfrak{J}]$, there exists a function $f(b_A, \delta)$ returning an integer range $[i_1, i_2]$ for a range δ over an attribute A with quantization scale $b_A \in \mathfrak{B}_{\mathfrak{J}}$, such that for any $Q \in \mathcal{Q}[\mathfrak{J}]$ and every selection predicate $A \mapsto \delta$ in $\text{Preds}_{\mathcal{Q}}^{\delta} : x \in \delta \Leftrightarrow b_A(x) \in f(b_A, \delta)$.*

As an example, Figure 2 shows the grid over table *mc* induced by two queries. The quantization scales $b_{\text{company_type_id}}$ and $b_{\text{company_id}}$ respectively correspond to the two *mc* attributes referenced by the queries' selection predicates. (For now, ignore the mention of “projected subslice,” which will be formally introduced in Section 3.4.) The buckets for $b_{\text{company_type_id}}$, numbered 1 through 3, are $[a_1, a_2]$, $[a_2, \text{succ}(a_2))$, and $[\text{succ}(a_2), \infty)$, where $\text{succ}(a_2)$ denotes the successor value of a_2 in the domain; any value in $(-\infty, a_1)$ will be quantized to special bucket id 0 because this range is not contained in any selection predicate. The buckets for $b_{\text{company_id}}$ are similarly induced by selection predicates involving *company_id*.

3.3 Quantized Selection Count Maps

Since queries in $\mathcal{Q}[\mathfrak{J}]$ ultimately only care about counts, a natural idea following selection attribute quantization is to further aggregate the points that fall into each grid cell induced by $\mathfrak{B}_{\mathfrak{J}}$ into a single count, instead of enumerating them.

Given a table $R \in \text{Tables}_{\mathfrak{J}}$ with selection attributes $\{A_1, \dots, A_k\}$ and quantization scales $\mathfrak{B}_{\mathfrak{J}}[R] = \{b_{A_1}, \dots, b_{A_k}\}$, we compress a subset of tuples in R into a (quantized selection) count map $\mathfrak{M}$: each entry of $\mathfrak{M}$ maps a grid coordinate $\vec{b} = (b_1, \dots, b_k)$ to $\mathfrak{M}[\vec{b}]$, the count of tuples within the grid cell—i.e., any tuple t satisfying $b_{A_i}(t.A_i) = b_i$ for $i = 1, \dots, k$. For example, Figure 2 shows the count map for the set of tuples (2D points in the geometric view).

We generalize the concept of count map $\mathfrak{M}$ over any subset of tables $\mathcal{R} \subseteq \text{Tables}_{\mathfrak{J}}$. Let $\text{Attrs}_{\mathfrak{J}}^{\mathcal{R}}[\mathcal{R}] = \text{Attrs}_{\mathfrak{J}}^{\mathcal{R}} \cap \bigcup_{R \in \mathcal{R}} \text{Attrs}[R]$ denote the set of selection attributes in $\mathcal{R}$ from all queries of pattern $\mathfrak{J}$. A grid coordinate for $\mathfrak{M}$, with one component for each attribute in $\text{Attrs}_{\mathfrak{J}}^{\mathcal{R}}$, identifies a grid cell induced by quantization scales $\{b_A \mid A \in \text{Attrs}_{\mathfrak{J}}^{\mathcal{R}}\}$ in subspace $\mathbb{S}[\text{Attrs}_{\mathfrak{J}}^{\mathcal{R}}]$. Given a subset T of tuples in

the cross product of $\mathcal{R}$ corresponding to points in $\mathbb{S}[\text{Attrs}_{\mathfrak{J}}^{\mathcal{R}}]$, $\mathfrak{M}$ counts the corresponding points of T in each grid cell.

Our goal is to construct a count map over the entire $\text{Tables}_{\mathfrak{J}}$ for the full join result set J , but importantly, without enumerating J . Intuitively, count maps represent intermediate results compactly. Obtaining a count map for a single table is easy, but combining count maps for different subsets of tables into a big one is complicated by joins, because not all points in two grid cells in orthogonal subspaces join with each other. However, leveraging Lemma 3.1, we can process points in J in groups: points in each group all share appropriate join attribute values, thereby allowing count maps to be computed for different subsets of tables and then conveniently “multiplied.” Then, the product count maps across groups can be “added” to obtain the final result count map. An example will be provided in Section 3.4.

We formally define “multiply” and “add” as follows:

- ⊗ **(multiply)** $\mathfrak{M}_1$ and $\mathfrak{M}_2$ are count maps over disjoint subsets $\mathcal{R}_1$ and $\mathcal{R}_2$ of tables in $\text{Tables}_{\mathfrak{J}}$. We define $\mathfrak{M}_1 \otimes \mathfrak{M}_2$, a count map over $\mathcal{R}_1 \cup \mathcal{R}_2$, as $\{\vec{b}_1 \sim \vec{b}_2 \mapsto (\mathfrak{M}_1[\vec{b}_1] \cdot \mathfrak{M}_2[\vec{b}_2]) \mid \vec{b}_1 \in \mathfrak{M}_1, \vec{b}_2 \in \mathfrak{M}_2\}$, where $\sim$ concatenates grid coordinate vectors.
- ⊕ **(add)** $\mathfrak{M}_1$ and $\mathfrak{M}_2$ are count maps over the same subset of tables $\mathcal{R} \subseteq \text{Tables}_{\mathfrak{J}}$ (and thus same quantization scales). We define $\mathfrak{M}_1 \oplus \mathfrak{M}_2$, a count map over $\mathcal{R}$, as $\{\vec{b} \mapsto (\mathfrak{M}_1[\vec{b}] + \mathfrak{M}_2[\vec{b}]) \mid \vec{b} \in \mathfrak{M}_1\}$.

The following lemmas establish the correctness of using $\otimes$ and $\oplus$ for computing count maps. Formally, given $\mathcal{Q}[\mathfrak{J}]$, quantization scales $\mathfrak{B}_{\mathfrak{J}}$, a subset of tables $\mathcal{R} \subseteq \text{Tables}_{\mathfrak{J}}$, and a subset T of tuples in the cross product of $\mathcal{R}$, we say that a count map $\mathfrak{M}$ over $\mathcal{R}$ is *complete* with respect to T if for any query $Q \in \mathcal{Q}[\mathfrak{J}]$, the size of the intersection between T and the selection-join subquery of Q restricted³ to $\mathcal{R}$ can be computed from $\mathfrak{M}$ and $\mathfrak{B}_{\mathfrak{J}}$.

LEMMA 3.3 (MULTIPLYING COUNT MAPS). *Given $\mathcal{Q}[\mathfrak{J}]$, consider disjoint subsets $\mathcal{R}_0, \dots, \mathcal{R}_n$ of tables in $\text{Tables}_{\mathfrak{J}}$, where $\text{Preds}_{\mathfrak{J}}^{\mathcal{R}_0}$ implies a join condition θ_i relating $\mathcal{R}_0$ to $\mathcal{R}_i$ for each $i = 1, \dots, n$, but there is no join condition across $\mathcal{R}_1, \dots, \mathcal{R}_n$ that is not already implied by $\bigwedge_i \theta_i$. For each $i = 1, \dots, n$, let E_i denote the join subquery of tables in $\mathcal{R}_i$, with condition implied by $\text{Preds}_{\mathfrak{J}}^{\mathcal{R}_0}$. Consider the star-shaped join query centered at E_0 :*

$$E_0 \bowtie_{\theta_1} E_1 \bowtie_{\theta_2} \dots \bowtie_{\theta_n} E_n,$$

which conforms to the query structure in Lemma 3.1. Denote by $A_0 = \text{Attrs}_{\mathfrak{J}}^{\mathcal{R}_0}[\mathcal{R}_0 \mid \bigcup_{i \in [1, n]} \mathcal{R}_i]$ the set of join attributes from E_0 referenced by $\theta_1, \dots, \theta_n$. Let v denote any mapping of every attribute $A \in A$ to a value $v(A)$ in A 's domain, for some attribute set A where $A_0 \subseteq A \subseteq \bigcup_{R \in \mathcal{R}_0} \text{Attrs}[R]$. Suppose:

- $\mathfrak{M}_0$ is a count map over E_0 complete w.r.t. $\sigma_{\bigwedge_{A \in A} A=v(A)} E_0$; and
 - $\forall i = 1, \dots, n$: $\mathfrak{M}_i$ is a count map over E_i complete w.r.t. $\sigma_{v[\theta_i]} E_i$.
- Then, $\mathfrak{M}_0 \otimes \mathfrak{M}_1 \otimes \dots \otimes \mathfrak{M}_n$ is complete with respect to*

$$\sigma_{\bigwedge_{A \in A} A=v(A)} (E_0 \bowtie_{\theta_1} E_1 \bowtie_{\theta_2} \dots \bowtie_{\theta_n} E_n).$$

LEMMA 3.4 (ADDING COUNT MAPS). *Given $\mathcal{Q}[\mathfrak{J}]$ and a subset of tables $\mathcal{R} \subseteq \text{Tables}_{\mathfrak{J}}$, suppose $\mathfrak{M}_1, \dots, \mathfrak{M}_n$ are count maps over $\mathcal{R}$,*

³More precisely, we remove from Q any table outside $\mathcal{R}$; remove from any equivalent class in $\text{Preds}_{\mathfrak{J}}^{\mathcal{Q}}$ any attribute outside $\mathcal{R}$ and then remove any empty or singleton equivalent class; and remove any entry in $\text{Preds}_{\mathfrak{J}}^{\mathcal{Q}}$ for an attribute outside $\mathcal{R}$.

Algorithm 1 BACon($\mathcal{Q}, \mathcal{D}$)

Input: A set $\mathcal{Q}$ of counting selection-join queries over $\mathcal{D}$.

Output: Result count for each query in $\mathcal{Q}$.

- 1: Scan $\mathcal{Q}$ and partition it into $\bigcup_{\mathfrak{J}} \mathcal{Q}[\mathfrak{J}]$ by join pattern;
 - 2: **for** each subset $\mathcal{Q}[\mathfrak{J}]$ of $\mathcal{Q}$ **do**
 - 3: $\mathfrak{B}_{\mathfrak{J}} \leftarrow \{\text{QUANTSCALES}(R, \mathcal{Q}[\mathfrak{J}]) \mid R \in \text{Tables}_{\mathfrak{J}}\}$;
 - 4: Determine the plan tree for $\mathfrak{J}$;
 - 5: $\mathfrak{M} \leftarrow \text{BAConRECURSE}(\text{root}(\mathfrak{J}), \emptyset)$;
 - 6: **yield from** COMPUTECOUNTS($\mathcal{Q}[\mathfrak{J}], \mathfrak{M}$);
 - 7: **end for**
-

where each $\mathfrak{M}_i$ is complete with respect to a subset T_i of tuples in the cross product of $\mathcal{R}$. If $T_1, \dots, T_n$ are disjoint, then $\mathfrak{M}_1 \oplus \dots \oplus \mathfrak{M}_n$ is complete with respect to $\bigcup_{i \in [1, n]} T_i$.

3.4 Basic BACon Algorithm

We are now ready to describe the overall BACon algorithm (Algorithm 1). First, we make a pass over all queries and partition them into subset where each contains queries with the same join pattern. For each subset $\mathcal{Q}[\mathfrak{J}]$ with join pattern $\mathfrak{J}$, we construct the quantization scales for them, as discussed in Section 3.2. We then determine an “execution plan” for $\mathfrak{J}$ as an ordered tree, akin to the notion of “(paths in) f-tree” in factorized database literature [8, 56]. Nodes in this tree correspond to the tables of $\text{Tables}_{\mathfrak{J}}$, and edges connecting the nodes represent equijoin conditions among them. Assuming that queries are acyclic and contain no cross products, one such tree always exists to capture $\text{Preds}_{\mathfrak{J}}^{\infty}$ precisely. Indeed, there can be multiple alternative plan trees; we defer the discussion of how to choose one to Section 4. At a high level, we process $\mathcal{Q}[\mathfrak{J}]$ using an in-order traversal of the plan tree starting from its root, by calling its main workhorse, the recursive procedure BAConRECURSE (Line 5), to compute a count map for $\mathcal{Q}[\mathfrak{J}]$. This count map is then used to calculate the result counts for queries, which can be done naively by summing up the counts at grid coordinates in each query hyperrectangle; we describe an optimized implementation in Section 4.

BAConRECURSE (Algorithm 2, illustrated in Figure 3) is called on a plan tree node (table) R and a filter specified by a mapping u that binds some attributes of R to specific values, which defines a “slice,” or the subset T of result tuples, of the join of subtree(R) (set of tables in the subtree rooted at R) satisfying u . The goal of BAConRECURSE is to compute a count map for this slice T (the call to root($\mathfrak{J}$) has $u = \emptyset$, so its slice in fact contains all result tuples in the full join of $\text{Tables}_{\mathfrak{J}}$). As discussed in Section 3.3, processing the entirety of T efficiently is hard; instead, BAConRECURSE partitions the slice into “subslices,” where each subslice T_v binds a particular combination v of values for R ’s attributes that join with its children, i.e., $\text{Attrs}_{\mathfrak{J}}^{\infty}[R \mid \text{children}(R)]$. To this end, BAConRECURSE calls PROCESSTABLE (Listing 3) to enumerate all possible such bindings (Line 2). Each iteration of the loop (Lines 2–10) computes a count map for the subslice T_v defined by a particular v . By Lemma 3.3, this count map can be computed by projecting T_v into “projected subslices,” one for R and each of R ’s child subqueries, and multiplying the count maps of these projected subslices. Conveniently, the call to PROCESSTABLE computes, for each projected subslice of R , the

Algorithm 2 BAConRECURSE(R, u)

Input: Mapping u binds attributes a subset of R ’s attributes to specific values. Implicitly, the function also has access to $\mathcal{D}$, $\mathfrak{J}$ and its plan tree, and the quantization scales $\mathfrak{B}_{\mathfrak{J}}$.

Output: A count map $\mathfrak{M}$ over subtree(R), complete w.r.t. result tuples of $\mathcal{Q}[\mathfrak{J}]$ restricted to subtree(R) and consistent with u .

- 1: $\mathfrak{M} \leftarrow \emptyset$; ▶ missing entries in all maps default to count 0
 - 2: **for** each subsequence $S[v]$ of entries $\langle v, \cdot, \cdot \rangle$ with the same v , returned by PROCESSTABLE($R, u, \text{Attrs}_{\mathfrak{J}}^{\infty}[R \mid \text{children}(R)]$) **do**
 - 3: $\mathfrak{M}_0 \leftarrow \{b \mapsto c \mid \langle v, b, c \rangle \in S[v]\}$;
 - 4: **for** each table $R_i \in \text{children}(R)$ **do**
 - 5: $u_i \leftarrow \left\{ A' \mapsto v(A) \mid \begin{array}{l} A \in \text{Attrs}_{\mathfrak{J}}^{\infty}[R \mid R_i] \wedge A' \in \text{Attrs}_{\mathfrak{J}}^{\infty}[R_i \mid R] \\ \wedge \text{Preds}_{\mathfrak{J}}^{\infty} \Rightarrow (A = A') \end{array} \right\}$;
 - 6: $\mathfrak{M}_i \leftarrow \text{BAConRECURSE}(R_i, u_i)$;
 - 7: $\mathfrak{M}_0 \leftarrow \mathfrak{M}_0 \otimes \mathfrak{M}_i$;
 - 8: **end for**
 - 9: $\mathfrak{M} \leftarrow \mathfrak{M} \oplus \mathfrak{M}_0$;
 - 10: **end for**
 - 11: **return** $\mathfrak{M}$;
-

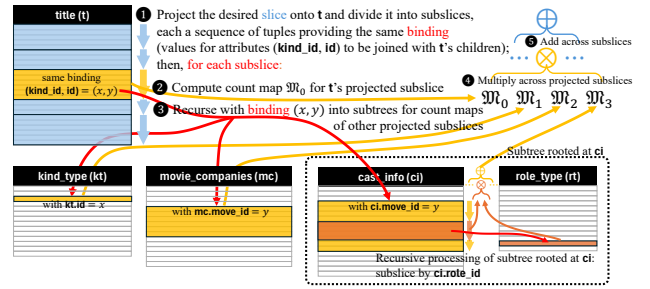


Figure 3: Illustration of Algorithm 2, continuing Figure 1.

coordinate-count pairs to populate the count map. We recursively call BAConRECURSE on each child of R to compute the count map for its projected subslice. Line 7, using $\otimes$, combines these count maps into a single count map for T_v . Finally, applying Lemma 3.4, Line 9 uses $\oplus$ to accumulate the count maps for all T_v ’s into the final count map for T to be returned.

EXAMPLE 3.2. Following Figure 3, at the root *title*, BACon iterates over all possible bindings for $(t.\text{kind_id}, t.\text{id})$ in turn. Suppose the blue subslice fixes $t.\text{kind_id} = 1, t.\text{id} = 1$. For the subslice projected to t , suppose $\mathfrak{M}_0 = \mathfrak{M}_t = \{(9) \mapsto 2, \dots\}$. We recursively call each child with the corresponding binding, as indicated by the arrows pointing to each child. For the “projected subslices” on children, we further assume that $\mathfrak{M}_2 = \mathfrak{M}_{mc}$ is the one shown in Figure 2. From the geometric view, $\mathfrak{M}_2$ is computed by the flattened hyperrectangle (in Figure 2) with $mc.\text{movie_id} = t.\text{id} = 1$. For illustration, let us say that $\mathfrak{M}_1 (\mathfrak{M}_{kt}) = \{\dots, (1) \mapsto 1, \dots\}$ and $\mathfrak{M}_3 (\mathfrak{M}_{ci\text{info}}) = \{\dots, (3, 3, 3) \mapsto 3, \dots\}$.

This subslice contributes to the global count map $\mathfrak{M}$ as follows: $\mathfrak{M} \leftarrow \mathfrak{M} \oplus (\mathfrak{M}_0 \otimes \dots \otimes \mathfrak{M}_3)$. We zoom in on one example—how the global count map entry at grid coordinate $(9, 1, 2, 2, 3, 3, 3)$ is updated. Concretely, this global grid coordinate is the concatenation of (9) from $\mathfrak{M}_0$, (1) from $\mathfrak{M}_1$, $(2, 2)$ from $\mathfrak{M}_2$, and $(3, 3, 3)$ from $\mathfrak{M}_3$. Recall that the counts in $\mathfrak{M}_0, \dots, \mathfrak{M}_3$ at these coordinates are 2, 1, 2, 3, respectively; therefore, the contribution to the global count map entry

Listing 3: SQL code for $\text{PROCESSTABLE}(R, u, \mathcal{A}^{\text{out}})$.

Input: Mapping u binds attributes $J_1^{\text{in}}, J_2^{\text{in}}, \dots$ to specific values, and $\mathcal{A}^{\text{out}} = \{J_1^{\text{out}}, J_2^{\text{out}}, \dots\}$ specifies the set of attributes for partitioning result entries. The quantization scales are $\mathcal{B}_3[R] = \{b_{S_1}, b_{S_2}, \dots\}$.

Output: Result entries have the form $\langle v, b, c \rangle$, sorted by v , where v is a mapping from $\mathcal{A}^{\text{out}}$ to values, b is a grid coordinate for $\mathcal{B}_3[R]$, and c is the associated count.

```
SELECT J_1^{\text{out}}, J_2^{\text{out}}, \dots, B1, B2, \dots, COUNT(*)
FROM (
  SELECT J_1^{\text{out}}, J_2^{\text{out}}, \dots,
    quantize(S_1, b_{S_1}) AS B1, quantize(S_2, b_{S_2}) AS B2, ...
  FROM R
  WHERE J_1^{\text{in}} = u(J_1^{\text{in}}) AND J_2^{\text{in}} = u(J_2^{\text{in}}) AND ...
) AS TMP
GROUP BY J_1^{\text{out}}, J_2^{\text{out}}, \dots, B1, B2, ...
ORDER BY J_1^{\text{out}}, J_2^{\text{out}}, \dots;
```

is $2 \times 1 \times 2 \times 3 = 12$. After processing this subslice, BACONRECURSE proceeds to the next $(t.\text{kind_id}, t.\text{id})$ combination in order.

Delving into PROCESSTABLE (Listing 3), we see how a single SQL query over a table R performs “subslicing” (by $\mathcal{A}^{\text{out}}$) and computes the count maps of all projected subslices. In the FROM subquery that defines TMP, the WHERE condition applies the attribute binding u that defines T , the slice of interest; the SELECT clause uses a user-defined function `quantize` to check each selection attribute value against a quantization scale and returns its grid coordinate. The outer query groups the TMP tuples by $\mathcal{A}^{\text{out}}$ to perform subslicing, and then by their grid coordinates to compute tuple count per grid cell for each projected subslice. Finally, ORDER BY ensures that entries for the same projected subslice, i.e., those with the same binding v for $\mathcal{A}^{\text{out}}$, are consecutive in the output. This ordering allows the caller BACONRECURSE to detect where a sequence of entries with a new v starts, so it can start processing a new subslice.

4 ADDITIONAL OPTIMIZATION AND IMPLEMENTATION DETAILS

This section fills in some of the details about BACON not covered by Section 3 and describes additional optimizations needed to make BACON practical for query workloads for training learned CE models. Before delving into details, we note that BACON takes the high-level approach of leveraging DBMS for processing, not only because it houses the data, but also because it already provides versatile support for indexing and querying. To make BACON easy to adopt, we do not modify any DBMS internals but instead implement, in a client application, parts of the algorithm that are inefficient for the DBMS. Many optimizations thus involve balancing database- and client-side processing options and mitigating the overhead of interfacing them. Overall, we perform PROCESSTABLE (Listing 3) inside the DBMS, using cursors to fetched join values, quantization grid coordinates, and associated counts. The remainder of BACONRECURSE (i.e., merging of count maps along the tree structure) and the remainder of BACON (i.e., COMPUTECOUNTS) are performed on the client side. The associated optimizations are presented below. While BACON is currently implemented on top of PostgreSQL, the ideas in this section generalize to other DBMSs, although details may differ.

Note that BACON is not optimized for join patterns characterized by queries exhibiting little opportunity for shared processing (e.g., when there are too few of them, or their predicates overlap

very little), or an underlying full join that is small and cheap to compute to begin with. In such cases, BACON may not outperform baselines INDPROC or POSTFILT . Therefore, we also have developed a method called HYBRID , which acts as an optimizer to choose the appropriate method to use among INDPROC , POSTFILT , and BACON for a given join pattern. In our experimental comparison of these methods in Section 5, we empirically analyze the cases where BACON is not optimal (Section 5.2): they follow our characterizations above and play only a small role in the overall performance of query workloads found in the learned CE literature. Furthermore, despite HYBRID ’s ability to make on per-pattern decisions, it offers no consistent improvement over BACON overall. This observation speaks to the effectiveness of BACON , even without HYBRID , for handling practical CE training workloads. More details on HYBRID can be found in the full version [42].

4.1 Choosing Execution Plan for a Join Pattern

Give a join pattern $\mathfrak{J}$, the cost of BACONRECURSE to process $\mathcal{Q}[\mathfrak{J}]$ depends on the choice of a tree-based execution plan, introduced in Section 3.4. A cost-based decision requires fine-grained knowledge of the data distribution, which itself is expensive to acquire. Instead, we follow some heuristics below to construct a tree-based plan.

The first heuristic is to pick tree root to be the table R with the highest degree in $\mathfrak{J}$ ’s join graph: i.e., the root joins with the largest number of other tables in $\text{Tables}_{\mathfrak{J}}$ according to $\text{Preds}_{\mathfrak{J}}^{\text{pk}}$. The rationale is to maximize the saving opportunities identified by Lemma 3.3: a root with a large in-degree can potentially avoid enumerating more cross products. Geometrically, this heuristic ensures that, during recursion, the grid coordinates in each descendant lie in low-dimensional subspaces. Once we pick the root, the rest of the tree structure follows naturally from $\text{Preds}_{\mathfrak{J}}^{\text{pk}}$. Since we assume queries to be acyclic, no additional join conditions are needed beyond those corresponding to the tree edges. For example, in Figure 1, even though `mc` and `ci` can join on `mc.movie_id = ci.movie_id`, there is no edge between them in the tree, because the two join conditions from root t to mc and ci , involving $t.\text{id}$, together imply `mc.movie_id = ci.movie_id`.

The next optimization then marks certain tree edges that correspond to joining a foreign key (FK) of the parent table R to a primary key (PK) of the child table R' . For such a marked FK-PK edge, when BACONRECURSE processes R , it will simultaneously process R' : on Line 2, PROCESSTABLE will additionally query R' and compute its count map along with that for R (with appropriate changes to Listing 3); and on Line 4, children of R' will be directly included, instead of R' itself. The justification behind this optimization is that, because of the FK-PK join, each v would yield only one joining tuple from R' , meaning there is no cross product to avoid in the first place. Therefore, we should “short-cut” this edge to eliminate the associated overhead.⁴ For simplicity, our implementation currently restricts this optimization to edges incident to the root, but applying it to the entire tree is possible as future work.

The last heuristic dynamically reorders subtrees under each node. We start with an arbitrary ordering. For a node R , after some iterations of the loop on Lines 2–10 of Algorithm 2, we count, for

⁴Note that the direction is important: a PK-FK edge, where the parent table has the primary key, would be a bad candidate to “short-cut,” with an opposite argument.

each child R_i , how many settings of v yield no tuples from R_i . Then, we reorder the children so that those with more such cases come first. Intuitively, they are more likely to produce an empty count map, with which we can “short-circuit” the sequence of multiplies.

4.2 Mitigating SQL Querying Overhead

Server- vs. Client-Side Cursors. When BACON runs a SQL query on the underlying database, we can use either a server-side cursor [68] or a client-side one [66]. A client-side cursor fetches the entire query result set into client memory. A server-side cursor, in contrast, allows the client to iterate row-by-row without materializing a potentially massive result set. While server-side cursors offer more flexibility and improve scalability, they are considerably slower and require state to be maintained on the database server; a large number of concurrent server-side cursors can cause performance issues. Therefore, BACON uses a server-side cursor for `PROCESSTABLE` on the root, but switches to client-side cursors for all other calls. The rationale is that the call on the root table (from Line 5 of Algorithm 1) has no bound attributes ($u = \emptyset$) and may generate a large result set. In contrast, other calls access much fewer tuples per subslice, and even with the batching optimization below, their result sizes remain manageable for client-side cursors.

Batching of `PROCESSTABLE`. During recursive execution of BACONRECURSE over the plan tree, the overhead of issuing many SQL queries via `PROCESSTABLE` can accumulate, especially when many of these queries, with specific attribute bindings, return small result sets. To mitigate this overhead, we batch multiple invocations of `PROCESSTABLE` for the same table R , using a set of bindings U rather than a single binding u , into one SQL query. Specifying the combined condition precisely in the WHERE clause of Listing 3 is generally infeasible, as it would have to enumerate all bindings in U . Instead, we compute the minimum and maximum elements⁵ of U and use $[\min(U), \max(U)]$ as a safe but possibly imprecise range bound in WHERE, i.e., WHERE ($J_1^{\text{in}}, J_2^{\text{in}}, \dots$) BETWEEN $\min(U)$ AND $\max(U)$. We extend the query to additionally group by $J_1^{\text{in}}, J_2^{\text{in}}, \dots$ and return them, such that `PROCESSTABLE` can, using these attribute values, filter out result tuples that do not belong to U .

In BACONRECURSE, we implement this optimization by collecting β consecutive settings of v into a set V for processing as a batch, analogous to a β -fold loop unrolling on Lines 2–10 of Algorithm 2. For each child R_i , we project V to obtain the set U_i of unique bindings on its join attributes with R (Line 5), and invoke `PROCESSTABLE` on R_i with U_i . The detailed algorithm is presented in the full version [42].

We now provide further analysis of this batching optimization. First, beyond reducing the overhead of issuing many queries, batching promotes reuse of `PROCESSTABLE` calls on the same table and attribute binding. Without batching, consecutive settings of v in the loop on Lines 2–10 of Algorithm 2: it is likely that they agree on most components since they are sorted. Therefore, when v is projected to obtain some u_i , the same u_i setting in the previous iteration may be used again to call `PROCESSTABLE` on the same table R_i , leading to waste unless we cache the results from previous calls. With batching, however, such reuse occurs naturally. So we

⁵If each binding involves multiple attributes, we order the bindings lexicographically.

have not implemented additional caching mechanisms for BACON, although further exploration will be a good future work.

Second, compressing a set of bindings into a single range may introduce false positives. Indeed, R_i may always contain some tuples whose join attribute values lie in $[\min(U_i), \max(U_i)]$ but are not contained in U_i , because U_i is generated from V and subject to all other constraints on the slice. Nonetheless, the concern with false positives is at least partly alleviated by the fact that BACONRECURSE processes settings of v in order, meaning that it will call `PROCESSTABLE` on R_i with “clustered” settings of u in U .

Implementation of quantize. Since the UDF `quantize` in Listing 3 is invoked many times, efficient implementation is crucial for performance. We implement `quantize` as C-language UDFs in PostgreSQL [1], with one variant for each SQL data type. The quantization scale is implemented as an array of sorted boundary points, and a binary search is used to determine the correct bucket id. These C-language UDFs run as compiled code natively inside the server process, greatly outperforming SQL-based UDFs. BACON creates these extensions at the beginning of its execution and drops them at the end. More generally, `quantize` can be implemented in other DBMSs using available extensions, including CLR (common language runtime) integration for Microsoft SQL Server [49] or external procedures for Oracle Database [57].

4.3 Implementation of `COMPUTECOUNTS`

`COMPUTECOUNTS` is invoked by BACON to compute the result counts of queries in $\mathcal{Q}[\mathcal{J}]$ given the final count map $\mathfrak{M}$ computed by BACONRECURSE. Conceptually, it performs a “join” between a set of query hyperrectangles ($\mathcal{Q}[\mathcal{J}]$) and a set of weighted points ($\mathfrak{M}$), and reports the total weights within each hyperrectangle. Many processing strategies are possible, and the best depends on query and data distributions. For our target of query workloads for training learned CE models, we observe that $|\mathcal{Q}[\mathcal{J}]|$ is moderate (in hundreds or thousands) and $\mathfrak{M}$ tends to be sparse (32% on average across join patterns in JOB’s synthetic). After experimenting with several methods, we have found a simple method based on nested loops to be the most effective for this setting. Basically, for each $Q \in \mathcal{Q}[\mathcal{J}]$ and for each non-zero entry $\bar{b} \mapsto c$ of $\mathfrak{M}$, we check whether, for every dimension of Q ’s hyperrectangle with a range bound, the corresponding coordinate in $\bar{b}$ falls within this bound. If yes, we add the entry’s count c to Q ’s running total. While BACON is implemented in Python, we specifically use Numba’s just-in-time compilation [4] for `COMPUTECOUNTS`. With the help from Numba’s compiler optimizations, this simple implementation was able to beat more sophisticated methods.

5 EXPERIMENTS

We compare INDPROC, POSTFILT,⁶ and BACON. All algorithms are implemented in Python3, using `psycopg2` [67] to connect to PostgreSQL V16.8 with default configuration. All experiments are conducted on a Linux server with an Intel(R) Xeon(R) Gold 5215 CPU

⁶We have also evaluated an alternative implementation of POSTFILT based on materialized views. The details and results, reported in the full version [42], show that materialization introduces additional storage costs and preparation-time overhead, without providing overall performance benefits. Hence, Section 5 focuses on the current implementation of POSTFILT.

(40 logical cores, 2.50GHz), 256GB of main memory, and 1TB of disk storage. We set the batching parameter β (Section 4.2) to 50,000 for BACON for all experiments and present results for different β 's in the full version [42]. Codes is available in the GitHub repository [43].

We test three popular datasets: IMDB [35], STATS [18], and DSB [10] (of scaling factor 2). They are loaded and initialized using the provided scripts [11, 21, 34], and the algorithms use exactly the indices provided therein without creating any additional ones.

We evaluate the algorithms on nine publicly available workloads in Table 1. These workloads span a wide range of sizes, query templates, join structures, predicate types, and result sizes, and are widely used for training and evaluating cardinality estimation techniques [18]. Among the workloads, only dsb-grasp-20k is post-processed [44] from the source file [74]. Specifically, CE work, SeConCDF [75] and GRASP [73], have numerical/categorical definitions for several attributes that are originally defined as char(...) in DSB (e.g., cd.cd_education_status). As these transformations are not clearly specified, we remove the queries involving such attributes, and then randomly sample 20k queries to ensure that baselines complete within a reasonable time. We additionally construct three new workloads based on job-light, used exclusively in Section 5.3.

Running time is our main performance metric. For INDPROC, end-to-end running time is the sum over all queries. For POSTFILT and BACON, it includes pre-processing and time for each join pattern. To prevent baselines from running for too long, we cap INDPROC at 900s per query, and POSTFILT at 3,600s per join pattern; therefore, some baseline numbers are only lower bounds.

In terms of client-side memory usage, because BACON processes each join pattern separately, memory peaks at the single most complex join pattern in the workload. Across all nine workloads, the maximum memory footprint of BACON occurs for a join pattern in synthetic, at 1.7GB. We do not report memory usage for POSTFILT and INDPROC, as they executed entirely inside the database server.

Single-Threaded vs. Parallel Execution. Results below focus on single-threaded execution to isolate the algorithmic differences of BACON compared with the baselines, without the potentially confounding factor of parallelization. We disable parallelism in PostgreSQL by setting max_parallel_workers_per_gather to 0, and restrict all client-side code to a single thread. Besides this single-threaded setting, we have also evaluated all methods in a parallel setting. Under the parallel setting, INDPROC and POSTFILT fully leverage PostgreSQL's built-in parallelism; for BACON, we use a simple multi-process design in which each process handles a subset of join patterns (more sophisticated parallelization strategies for BACON are possible as future work). Briefly, the results show that, even with this simple strategy, BACON achieves end-to-end speedups of 2 $\times$ to 598 $\times$ compared with INDPROC, consistent with the single-threaded results shown below in Section 5.1. Additional details on the parallel setting can be found in the full version [42].

5.1 End-to-End Running Time

Table 2 reports the end-to-end running times of all three approaches across all nine real workloads. Lower-bound values (followed by "+") are reported if a baseline runs out of time. We additionally show the relative speedups of BACON over INDPROC, which range from approximately 2 $\times$ to 178 $\times$. Speedups over POSTFILT are omitted, as

Table 2: End-to-end running times (seconds, rounded) and relative speedups. +: lower-bound values, as INDPROC is capped at 900s per query and POSTFILT at 3,600s per join pattern.

Query Workload	INDPROC	POSTFILT	BACON	Speedup vs. INDPROC
synthetic	19,663	11,329+	1,574	$\uparrow 12.49 \times$
scale	21,100+	26,342+	3,475	$\uparrow 6.07 \times$
job-light	1,620+	13,662+	728	$\uparrow 2.22 \times$
job-light-single	249	13	11	$\uparrow 22.50 \times$
job-light-join	8,601+	18,353+	1,541	$\uparrow 5.58 \times$
stats-ceb	4,274+	47,353+	58	$\uparrow 73.62 \times$
stats-ceb-single	18	2	2	$\uparrow 8.70 \times$
stats-ceb-join	35,292+	53,363+	198	$\uparrow 178.14 \times$
dsb-grasp-20k	19,746	4,211	928	$\uparrow 21.28 \times$

INDPROC outperforms POSTFILT on most long-running workloads. Overall, BACON consistently outperforms both baselines and never times out: the worst per-join-pattern overhead of BACON across all workloads is below 380s, far below the thresholds for baselines. The observed performance trends between INDPROC and POSTFILT align with our analysis in Section 2. In particular, POSTFILT is advantageous for join patterns with many queries—where predicates tend to cover larger regions—and without intermediate join blow-ups (e.g., synthetic and dsb-grasp-20k). In the remaining cases, INDPROC performs substantially better due to selection push-down in each query. BACON is able to effectively combine the strengths of both INDPROC and POSTFILT.

Since both POSTFILT and BACON are designed around the notion of join patterns, we further analyze the results on a per-join-pattern basis in Section 5.2. Before doing so, we briefly summarize the other overheads—which are negligible compared with per-join-pattern running time—and omit them from ensuing discussions. INDPROC incurs no such overheads. Both POSTFILT and BACON require loading and parsing SQL queries, extracting join patterns, and grouping queries accordingly, with complexity linear in the number of queries, tables, and selection attributes. This pre-processing step happens at the beginning when processing each join pattern, and typically completes within a few seconds; the only exception is dsb-grasp-20k, where it reaches ≈ 20 s due to the large number of queries (but is still small relative to the end-to-end running time).

5.2 Per-Join-Pattern Result

In this section, we analyze the results on a per-join-pattern basis. Although INDPROC does not operate on join patterns, we group its results accordingly to facilitate direct comparison with the others.

We first report the number of timed-out cases in Table 3. BACON never times out. Overall, POSTFILT times out more frequently: since it always materializes the unfiltered join, if that times out, all queries associated with the pattern time out. Additionally, workloads on STATS exhibit more time-outs, primarily due to larger intermediate joins than those on IMDB, as noted by Han et al. [18].

Next, we analyze the per-join-pattern overhead across all workloads. For clarity, consistent with the pre-defined time-out thresholds, we assign 3,600s to timed-out join patterns in POSTFILT and 900s to timed-out queries in INDPROC; actual running times will be higher. Figure 4 reports the running times of all join patterns,

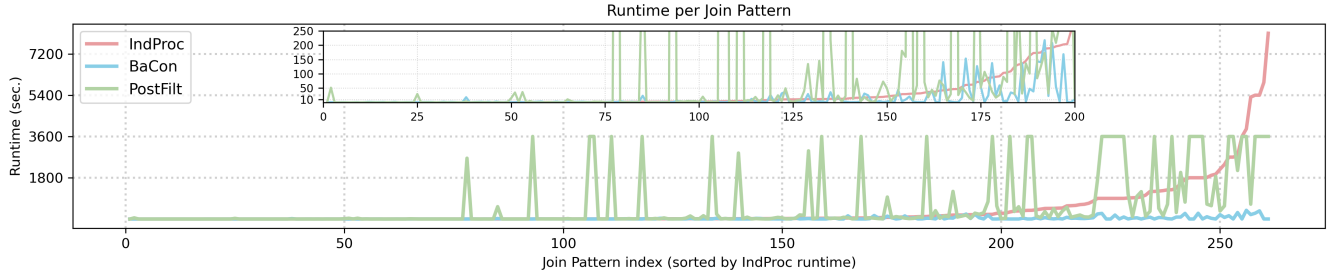


Figure 4: Running time (seconds) per join pattern across 283 join patterns from 9 workloads. Join patterns are ordered by INDPROC’s times. The inset is the zoom-in of the first 200 indices.

Table 3: Number of timed-out cases. In (a, b) , a is the number of timed-out queries, and b is the number of join patterns that contain any timed-out queries.

Query Workload	# queries	# join patterns	INDPROC	POSTFILT
synthetic	5,000	16	(0, 0)	(251, 1)
scale	500	31	(20, 7)	(84, 6)
job-light	70	18	(1, 1)	(8, 2)
job-light-single	254	1	(0, 0)	(0, 0)
job-light-join	696	27	(3, 1)	(32, 3)
stats-ceb	146	58	(3, 2)	(30, 11)
stats-ceb-single	632	1	(0, 0)	(0, 0)
stats-ceb-join	2,603	120	(22, 7)	(166, 13)
dsb-grasp-20k	20,000	16	(0, 0)	(0, 0)

ordered by INDPROC’s times. Before index 120, all join patterns under INDPROC complete within 10s; beyond this point, running times increase markedly. The inset zooms in the region before index 200, where BACon closely tracks INDPROC, sometimes faster and sometimes slower within a bounded margin, while POSTFILT already times out on several patterns. Between indices 160 and 200, BACon exhibits a small number of clear regressions relative to INDPROC, with the largest gap (98s) at index 165; we analyze these cases in detail later. Outside the zoomed region, where join patterns run substantially longer, BACon consistently and significantly outperforms INDPROC and POSTFILT.

Additionally, we present a detailed per-join-pattern analysis for scale and job-light here; results for the remaining workloads are in the full version [42]. As shown in Figure 5, scale contains join patterns involving 1 to 5 tables, with more tables generally run longer. Timed-out queries arise mainly from patterns with many tables. For most long-running join patterns, BACon consistently outperforms the baselines. For fast patterns, BACon remains competitive, as the time gaps are small. However, there are several patterns where BACon is noticeably slower than either baseline. Taking join pattern 6 as an example, INDPROC, POSTFILT, and BACon take 61s, 35s, and 153s, respectively. This pattern has conditions $ci.movie_id = t.id$ AND $mc.movie_id = t.id$ and only five queries. A breakdown of BACon’s running time shows 118s (76%) for executing SQLs, 5s (3%) for aggregating rows by join attribute bindings and coordinates, and 18s (15%) for merging count maps; thus, SQL

execution dominates and already exceeds the running times of the baselines. This regression stems from two factors. First, algorithmically, enumerating tuples from each table and quantizing them can be less efficient than INDPROC when predicates are selective and overlaps are limited. Second, although BACon performs less work than POSTFILT, BACon’s cursor invocations and row fetching/parsing incur more overhead than POSTFILT’s server-side processing.

We have similar observations from Figure 6. As discussed by Han et al. [18], job-light is a classical but relatively simple workload with 1 to 8 queries for each join pattern. The true cardinality range is also a magnitude smaller than that of stats-ceb. Therefore, INDPROC consistently performs well on all join patterns, except one severe time-out for join pattern 17. However, BACon remains competitive across join patterns and never times out. Regression compared with baselines is limited: in the worst case, BACon is 55s slower compared with INDPROC on join pattern 8, and 100s compared with POSTFILT on join pattern 7.

Overall, consistent with the results in Section 5.1, BACon does well across join patterns: it significantly outperforms the baselines on expensive patterns and remains acceptable on cheap ones.

5.3 Scalability

In this section, we evaluate the scalability of the three approaches using three synthetic workloads, denoted job-light-*. We choose job-light as the reference because, as shown in Figure 6, INDPROC performs consistently well, except for a single join pattern that times out. Specifically, on job-light, BACon outperforms INDPROC in only 7 out of the 18 (39%) join patterns. Earlier, we have made the observation that the baselines—especially INDPROC—can outperform BACon on simple join patterns, e.g., where the number of queries is small and intermediate join sizes remain moderate. In particular, if the number of queries is small, there is less opportunity for sharing, and INDPROC’s simple approach of optimizing and evaluating queries one by one works fine. Here, we further investigate how each approach scales as the workload size increases while preserving the original distribution.

We construct workloads with 1,000, 2,000, and 4,000 queries each. Queries are generated randomly and independently. We randomly select a query from job-light, reuse its template (and thus its join pattern), but modify its predicate constants: 1) for a constant in equality and inequality predicates, we replace it with a random

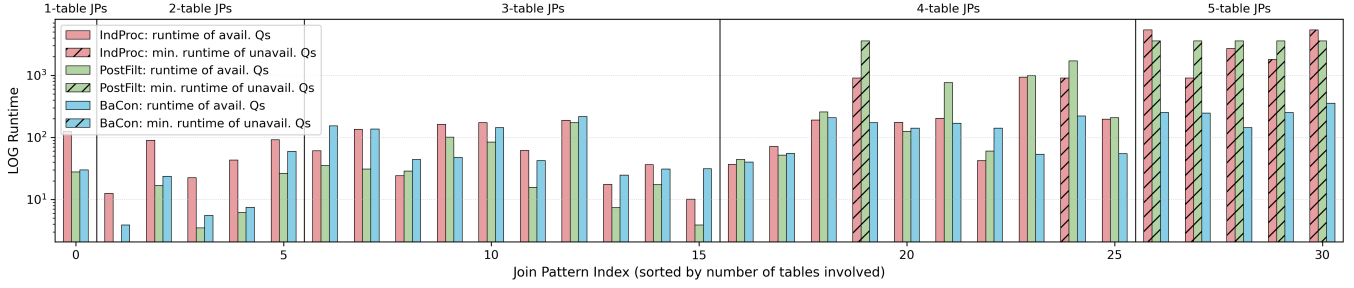


Figure 5: Log-scale running time per join pattern in scale. Join patterns are ordered by the number of tables involved, and the figure is partitioned accordingly (labels shown above). Hatched bars (baselines only) indicate the lower bounds for timed-out cases.

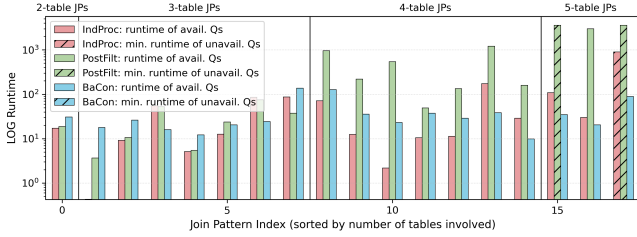


Figure 6: Log-scale running time per join pattern in job-light, with same format as Figure 5.

Table 4: End-to-end running times (seconds, rounded) and relative speedups. +: lower-bound values, as INDPROC is capped at 900s per query and POSTFILT at 3,600s per join pattern.

Query Workload	INDPROC	POSTFILT	BACON	Speedup vs. INDPROC
job-light-1k	6,208	22,490+	1,020	$\uparrow 6.09 \times$
job-light-2k	13,818	26,923+	1,051	$\uparrow 13.15 \times$
job-light-4k	23,617+	33,981+	1,109	$\uparrow 21.29 \times$

value from the active domain of the corresponding attribute; 2) for range predicates, we randomize the left endpoint and adjust the right accordingly to preserve the range length.

Table 4 shows end-to-end running times and relative speedups. As expected, INDPROC’s running time grows roughly linearly with workload size, since queries are processed independently. POSTFILT grows more slowly because its dominant cost—computing the full join—is amortized across queries with the same join pattern; nevertheless, it still times out frequently. BACON scales better for two reasons. First, quantization generally benefits larger workloads more. Second, the costs of enumerating join attribute bindings and quantization remain mostly stable: as the size of each quantization scale is bounded by the workload size $|\mathcal{Q}[\mathcal{S}]|$, the cost of quantizing an attribute is only $O(\log(|\mathcal{Q}[\mathcal{S}]|))$ thanks to binary search. Per-join-pattern results are provided in the full version [42]. While the number of join patterns remains 18, the number of them where BACON outperforms INDPROC increases to 14 (78%) in job-light-1k, 17 (94%) in job-light-2k, and finally 18 (100%) in job-light-4k. Overall, BACON demonstrates superior scalability compared with both INDPROC and POSTFILT.

Table 5: End-to-end (E2E) running times for BACON and HYBRID, with the number of join patterns handled by each algorithm. I/P/B denotes INDPROC/POSTFILT/BACON.

Query Workload	BACON E2E running time	HYBRID E2E running time	# join patterns using I/P/B
scale	3,475	3,319	0/9/22
stats-ceb	58	195	3/1/54
other 5	-	-	0/0/all

5.4 Validation against HYBRID

As shown in Section 5.2, there exist join patterns for which a baseline, INDPROC or POSTFILT, outperforms BACON. These cases typically involve few queries and small intermediate join sizes. As discussed in the full version [42], we can adopt a hybrid approach that uses a classifier to pick which method to use given a join pattern. Here, we train HYBRID using observations from job_light_join and stats_ceb_join, and evaluate it on the remaining seven workloads. Table 5 reports the E2E running times of HYBRID (using BACON as the reference) on the seven test workloads. Overall, HYBRID selects a baseline for 9 join patterns in scale and 4 join patterns in stats_ceb, while selecting BACON for all other join patterns and workloads. Even with HYBRID’s conservative design, end-to-end running times show that mispredictions can occur, sometimes catastrophically. To take a closer look, Figure 7 presents a per-join-pattern analysis, focusing on patterns where HYBRID chooses a baseline. As shown in Figure 7, for scale, all 9 switches to POSTFILT are beneficial, yielding a total saving of 156s. In stats_ceb, HYBRID gains minor savings by choosing POSTFILT on pattern 27 and INDPROC on pattern 57, but incorrectly selecting INDPROC on patterns 40 and 56 causes slowdowns of 1s and 129s, respectively. Overall, these results confirm that BACON remains a safe bet for practical workloads. With additional data statistics or training, a better HYBRID might be possible as future work, but the overhead for statistics collection and training may outweigh its benefit.

6 RELATED WORK

Select-Join-Aggregate Query Processing. Counting join results (with selection predicates on base tables) is a special form of select-join-aggregate queries. Standard processing typically involves pushing down selection predicates before evaluating the join and then

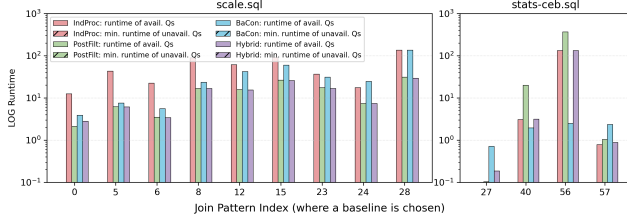


Figure 7: Log-scale running time per join pattern (where a baseline is chosen) in scale and stats_ceb, including HYBRID's.

aggregate. The classic Yannakakis algorithm [79] computes free-connex join-count queries in $O(N)$ time, where N is the number of tuples. Recent advancements [26] exploit a hybrid strategy of Yannakakis algorithm that handles acyclic join-count queries in $O(N + N \cdot \text{OUT}^{1-\frac{1}{w}})$ time, where OUT is the size of the projection of the join results onto the counting columns, and w is the free-connex fractional hypertree width of the query.

BACON draws inspiration from these techniques, specifically the recursive computation and merging of coordinate-to-count maps during backtracking from [2, 14, 48, 70]. Cost-based optimizers in these systems, such as Free Join's [70], together with the AGM bound [6, 16], further inform our initial exploration of cost-based hybrid mechanism detailed in the full version [42]. However, more accurate cost estimates may conversely require more expensive estimation methods, such as LpBound [80], and/or specialized data storages and execution engines, such as Free Join's COLT [70] and its integration to DuckDB [59].

Aggregation Push-down and Factorizations. Aggregation push-down reduces intermediate results via early or deferred aggregation. Classical approaches include eager and lazy aggregation [76] and integrating GROUP BY into cost-based optimization [9]. GuAo [33] identifies cases where join materialization can be avoided entirely and introduces a corresponding physical operator for SparkSQL [5]. Its propagation of frequencies grouped by join attributes resembles BACON's bucketization in single-query settings. However, it relies on binary joins, whereas our approach support multi-way merging of count maps. Factorized databases [7, 8, 56] eliminate tuple-level redundancy in query results through compact representations that can support efficient aggregation beyond COUNT. These works, focusing on eliminating intra-query redundancy, are complementary to our approach, which also exploits inter-query sharing. Furthermore, these works typically employ non-traditional storage layout different from traditional relational databases.

Optimized Batch Processing of Aggregate Queries. LMFAO [62] evaluates batches of ground-by aggregates over shared joins without materializing intermediates by decomposing queries into views, organizing them via a join tree, and executing a multi-output plan. Like POSTFILT, it encodes selection predicates as conditional expressions within aggregates. Unlike BACON, LMFAO does not analyze or pre-process overlaps among cross-query predicates, but instead focuses on sharing computation of join tree traversal and attribute-ordered evaluation. Moreover, LMFAO is implemented as a standalone execution engine.

Multi-Query Optimization & Shared Workload Optimization. Multi-query optimization (MQO) [65] identifies shared or similar subexpressions across queries and evaluate them jointly or via plan rewriting. Zhou et al. [81] propose a common subexpression manager integrated into Microsoft SQL Server and its cost-based query optimizer, demonstrating improvements on workloads of tens of queries. Shared Workload Optimization (SWO) [15] extends this idea to larger workloads by sharing operators, often requiring specialized execution engines. These approaches target general workloads and require accurate cost estimates.

Learned Cardinality Estimation. Query-aware cardinality estimation (CE) models [13, 27, 29, 37, 39, 40, 50, 51, 58, 71] learn to predict cardinalities from query-count pairs. Training is typically performed either in a single-shot manner, where a model is (re-)trained from scratch on a collected batch of queries [13, 27, 37, 38, 40], or in multiple rounds, where the model is incrementally updated, e.g., via fine-tuning, as new query-count pairs become available [25, 39, 58]. Some hybrid CE models [29, 39] additionally require data statistics, such as samples or histograms. Recent work studies robustness, training efficiency, and adaptivity to data updates and workload shifts [25, 32, 37, 39, 52, 60, 72, 75], increasing the demand for efficiently supporting counting queries.

More Discussion & Other Related Work. Our work is also related to research areas such as continuous query processing. A more detailed discussion is provided in the full version [42].

7 CONCLUSION

In this paper, we presented BACON, a method for efficient batch processing of counting queries. BACON brings together multiple optimization ideas, focusing particularly on developing compact, alternative representations of intermediate results that enable fine-grained sharing of computation. BACON combines lightweight SQL execution with client-side processing, incorporating a suite of optimizations to ensure practical performance on our target workloads. A strength of BACON is its practicality: it can be deployed without modifying DBMS internals or physical designs. Components of BACON can be accelerated using efficient UDF implementations (such as C-extensions in PostgreSQL), if they are supported by the DBMS. Our results demonstrate significant performance gains across real workloads with diverse characteristics. This level of improvement is empowering: shorter (re)training times make learned CE more practical and allow larger, more comprehensive training workloads.

BACON opens up several promising directions for future work. Extending it to support richer join structures, more expressive selection predicates, and more complex aggregates (complex expression inputs or non-algebraic aggregate functions) would further broaden its applicability. Beyond functional extensions, there are opportunities for more optimization, such as sharing computation across join patterns; delayed evaluation of $\oplus$ and $\otimes$ in count map expressions (and adapting COMPUTECOUNTS to take advantage); more intelligent selection of tree-based execution plans; aggressive "short-cutting" of all FK-PK edges; caching and more sophisticated parallelization. Finally, new applications and additional optimizations will likely require models beyond the current HYBRID to enable cost-based selection of processing methods.

REFERENCES

- [1] PostgreSQL 17. 2025. Extending SQL: C-Language Functions. <https://www.postgresql.org/docs/current/xfunc-c.html>
- [2] Christopher R. Aberger, Andrew Lamb, Susan Tu, Andres Nötzli, Kunle Olukotun, and Christopher Ré. 2017. EmptyHeaded: A Relational Engine for Graph Processing. *ACM Trans. Database Syst.* 42, 4, Article 20 (Oct. 2017), 44 pages. <https://doi.org/10.1145/3129246>
- [3] Pankaj K. Agarwal, Junyi Xie, Jun Yang, and Hai Yu. 2006. Scalable continuous query processing by tracking hotspots. In *Proceedings of the 32nd International Conference on Very Large Data Bases (Seoul, Korea) (VLDB '06)*. VLDB Endowment, 31–42.
- [4] Inc. Anaconda et al. 2025. Numba - a just-in-time compiler for Python that works best on code that uses NumPy arrays and functions, and loops. <https://numba.pydata.org/numba-doc/dev/index.html#>
- [5] Michael Armbrust, Reynold S. Xin, Cheng Lian, Yin Huai, Davies Liu, Joseph K. Bradley, Xiangrui Meng, Tomer Kaftan, Michael J. Franklin, Ali Ghodsi, and Matei Zaharia. 2015. Spark SQL: Relational Data Processing in Spark. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data (Melbourne, Victoria, Australia) (SIGMOD '15)*. Association for Computing Machinery, New York, NY, USA, 1383–1394. <https://doi.org/10.1145/2723372.2742797>
- [6] Albert Atserias, Martin Grohe, and Daniel Marx. 2017. Size bounds and query plans for relational joins. *arXiv:1711.03860 [cs.DB]* <https://arxiv.org/abs/1711.03860>
- [7] Nurzhan Bakibayev, Tomáš Kočíský, Dan Olteanu, and Jakub Závodný. 2013. Aggregation and ordering in factorised databases. *Proc. VLDB Endow.* 6, 14 (Sept. 2013), 1990–2001. <https://doi.org/10.14778/2556549.2556579>
- [8] Nurzhan Bakibayev, Dan Olteanu, and Jakub Závodný. 2012. FDB: a query engine for factorised relational databases. *Proc. VLDB Endow.* 5, 11 (July 2012), 1232–1243. <https://doi.org/10.14778/2350229.2350242>
- [9] Surajit Chaudhuri and Kyuseok Shim. 1994. Including Group-By in Query Optimization. In *Proceedings of the 20th International Conference on Very Large Data Bases (VLDB '94)*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 354–366.
- [10] Bailu Ding, Surajit Chaudhuri, Johannes Gehrke, and Vivek Narasayya. 2021. DSB: a decision support benchmark for workload-driven and traditional database systems. *Proc. VLDB Endow.* 14, 13 (Sept. 2021), 3376–3388. <https://doi.org/10.14778/3484224.3484234>
- [11] Bailu Ding, Surajit Chaudhuri, Johannes Gehrke, and Vivek Narasayya. 2021. DSB Initialization Files and Scripts. <https://github.com/microsoft/dsb/tree/main/scripts>
- [12] Lyric Doshi, Vincent Zhuang, Gaurav Jain, Ryan Marcus, Haoyu Huang, Deniz Altinbükten, Eugene Brevdo, and Campbell Fraser. 2023. Kepler: Robust Learning for Parametric Query Optimization. *Proc. ACM Manag. Data* 1, 1, Article 109 (May 2023), 25 pages. <https://doi.org/10.1145/3588963>
- [13] Anshuman Dutt, Chi Wang, Azade Nazi, Srikanth Kandula, Vivek Narasayya, and Surajit Chaudhuri. 2019. Selectivity estimation for range predicates using lightweight models. *Proc. VLDB Endow.* 12, 9 (May 2019), 1044–1057. <https://doi.org/10.14778/3329772.3329780>
- [14] Michael Freitag, Maximilian Bandle, Tobias Schmidt, Alfons Kemper, and Thomas Neumann. 2020. Adopting worst-case optimal joins in relational database systems. *Proc. VLDB Endow.* 13, 12 (July 2020), 1891–1904. <https://doi.org/10.14778/3407790.3407797>
- [15] Georgios Giannikis, Darko Makreshanski, Gustavo Alonso, and Donald Kossmann. 2014. Shared workload optimization. *Proc. VLDB Endow.* 7, 6 (Feb. 2014), 429–440. <https://doi.org/10.14778/2732279.2732280>
- [16] Georg Gottlob, Stephanie Tien Lee, Gregory Valiant, and Paul Valiant. 2012. Size and Treewidth Bounds for Conjunctive Queries. *J. ACM* 59, 3, Article 16 (June 2012), 35 pages. <https://doi.org/10.1145/2220357.2220363>
- [17] Jim Gray, Surajit Chaudhuri, Adam Bosworth, Andrew Layman, Don Reichart, Murali Venkatrao, Frank Pellow, and Hamid Pirahesh. 1997. Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals. *Data Min. Knowl. Discov.* 1, 1 (Jan. 1997), 29–53. <https://doi.org/10.1023/A:1009726021843>
- [18] Yuxing Han, Ziniu Wu, Peizhi Wu, Rong Zhu, Jingyi Yang, Liang Wei Tan, Kai Zeng, Gao Cong, Yanzhao Qin, Andreas Pfadler, Zhengping Qian, Jingren Zhou, Jiangneng Li, and Bin Cui. 2021. Cardinality estimation in DBMS: a comprehensive benchmark evaluation. *Proc. VLDB Endow.* 15, 4 (Dec. 2021), 752–765. <https://doi.org/10.14778/3503585.3503586>
- [19] Yuxing Han, Ziniu Wu, Peizhi Wu, Rong Zhu, Jingyi Yang, Liang Wei Tan, Kai Zeng, Gao Cong, Yanzhao Qin, Andreas Pfadler, Zhengping Qian, Jingren Zhou, Jiangneng Li, and Bin Cui. 2023. job-light-join.sql. https://github.com/Nathaniel-Han/End-to-End-CardEst-Benchmark/blob/master/workloads/job_light_sub_queries/job_light_sub_query.sql
- [20] Yuxing Han, Ziniu Wu, Peizhi Wu, Rong Zhu, Jingyi Yang, Liang Wei Tan, Kai Zeng, Gao Cong, Yanzhao Qin, Andreas Pfadler, Zhengping Qian, Jingren Zhou, Jiangneng Li, and Bin Cui. 2023. job-light-single.sql. https://github.com/Nathaniel-Han/End-to-End-CardEst-Benchmark/blob/master/workloads/job_light/sub_plan_queries/job_light_single_table_sub_query.sql
- [21] Yuxing Han, Ziniu Wu, Peizhi Wu, Rong Zhu, Jingyi Yang, Liang Wei Tan, Kai Zeng, Gao Cong, Yanzhao Qin, Andreas Pfadler, Zhengping Qian, Jingren Zhou, Jiangneng Li, and Bin Cui. 2023. STATS Initialization Files and Scripts. <https://github.com/Nathaniel-Han/End-to-End-CardEst-Benchmark/tree/master/scripts/sql>
- [22] Yuxing Han, Ziniu Wu, Peizhi Wu, Rong Zhu, Jingyi Yang, Liang Wei Tan, Kai Zeng, Gao Cong, Yanzhao Qin, Andreas Pfadler, Zhengping Qian, Jingren Zhou, Jiangneng Li, and Bin Cui. 2023. stats_ceb_join.sql. https://github.com/Nathaniel-Han/End-to-End-CardEst-Benchmark/blob/master/workloads/stats_CEB/sub_plan_queries/stats_CEB_sub_queries.sql
- [23] Yuxing Han, Ziniu Wu, Peizhi Wu, Rong Zhu, Jingyi Yang, Liang Wei Tan, Kai Zeng, Gao Cong, Yanzhao Qin, Andreas Pfadler, Zhengping Qian, Jingren Zhou, Jiangneng Li, and Bin Cui. 2023. stats_ceb_single.sql. https://github.com/Nathaniel-Han/End-to-End-CardEst-Benchmark/blob/master/workloads/stats_CEB/sub_plan_queries/stats_CEB_single_table_sub_query.sql
- [24] Yuxing Han, Ziniu Wu, Peizhi Wu, Rong Zhu, Jingyi Yang, Liang Wei Tan, Kai Zeng, Gao Cong, Yanzhao Qin, Andreas Pfadler, Zhengping Qian, Jingren Zhou, Jiangneng Li, and Bin Cui. 2023. stats_ceb.sql. https://github.com/Nathaniel-Han/End-to-End-CardEst-Benchmark/blob/master/workloads/stats_CEB/stats_CEB.sql
- [25] Mike Heddes, Igor Nunes, Tony Givargis, and Alex Nicolau. 2024. Convolution and Cross-Correlation of Count Sketches Enables Fast Cardinality Estimation of Multi-Join Queries. *Proc. ACM Manag. Data* 2, 3, Article 129 (May 2024), 26 pages. <https://doi.org/10.1145/3654932>
- [26] Xiao Hu. 2025. Output-Optimal Algorithms for Join-Aggregate Queries. *Proc. ACM Manag. Data* 3, 2, Article 104 (June 2025), 27 pages. <https://doi.org/10.1145/3725241>
- [27] Xiao Hu, Yuxi Liu, Haibo Xiu, Pankaj K. Agarwal, Debmalya Panigrahi, Sudeepa Roy, and Jun Yang. 2022. Selectivity Functions of Range Queries are Learnable. In *SIGMOD (Philadelphia, PA, USA) (SIGMOD '22)*. 959–972.
- [28] Andreas Kipf, Thomas Kipf, Bernhard Radke, Viktor Leis, Peter Boncz, and Alfons Kemper. 2018. job-light.sql. <https://github.com/andreaskipf/learnedcardinalities/blob/master/workloads/job-light.sql>
- [29] Andreas Kipf, Thomas Kipf, Bernhard Radke, Viktor Leis, Peter Boncz, and Alfons Kemper. 2018. Learned cardinalities: Estimating correlated joins with deep learning. *arXiv preprint arXiv:1809.00677* (2018).
- [30] Andreas Kipf, Thomas Kipf, Bernhard Radke, Viktor Leis, Peter Boncz, and Alfons Kemper. 2018. scale.sql. <https://github.com/andreaskipf/learnedcardinalities/blob/master/workloads/scale.sql>
- [31] Andreas Kipf, Thomas Kipf, Bernhard Radke, Viktor Leis, Peter Boncz, and Alfons Kemper. 2018. synthetic.sql. <https://github.com/andreaskipf/learnedcardinalities/blob/master/workloads/synthetic.sql>
- [32] Meghdad Kurmanji and Peter Triantafyllou. 2023. Detect, Distill and Update: Learned DB Systems Facing Out of Distribution Data. *Proc. ACM Manag. Data* 1, 1, Article 33 (May 2023), 27 pages. <https://doi.org/10.1145/3588713>
- [33] Matthias Lanzinger, Reinhard Pichler, and Alexander Selzer. 2025. Avoiding Materialization for Guarded Aggregate Queries. *Proc. VLDB Endow.* 18, 5 (Jan. 2025), 1398–1411. <https://doi.org/10.14778/3718057.3718068>
- [34] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2013–2019. IMDB Initialization Files and Scripts. <https://event.cwi.nl/da/job/>
- [35] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2015. How good are query optimizers, really? *Proc. VLDB Endow.* 9, 3 (Nov. 2015), 204–215. <https://doi.org/10.14778/2850583.2850594>
- [36] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2015. IMDB Relational Schema by JOB. <https://event.cwi.nl/da/job/>
- [37] Beibin Li, Yao Lu, and Srikanth Kandula. 2022. Warper: Efficiently Adapting Learned Cardinality Estimators to Data and Workload Drifts. In *Proceedings of the 2022 International Conference on Management of Data (Philadelphia, PA, USA) (SIGMOD '22)*. Association for Computing Machinery, New York, NY, USA, 1920–1933. <https://doi.org/10.1145/3514221.3526179>
- [38] Beibin Li, Yao Lu, Chi Wang, and Srikanth Kandula. 2021. Cardinality Estimation: Is Machine Learning a Silver Bullet?. In *AIDB*. <https://www.microsoft.com/en-us/research/publication/cardinality-estimation-is-machine-learning-a-silver-bullet/>
- [39] Pengfei Li, Wenqing Wei, Rong Zhu, Bolin Ding, Jingren Zhou, and Hua Lu. 2023. ALECE: An Attention-based Learned Cardinality Estimator for SPJ Queries on Dynamic Workloads. *Proc. VLDB Endow.* 17, 2 (Oct. 2023), 197–210. <https://doi.org/10.14778/3626292.3626302>
- [40] Jie Liu, Wenqian Dong, Qingqing Zhou, and Dong Li. 2021. Fauce: fast and accurate deep ensembles with uncertainty for cardinality estimation. *Proc. VLDB Endow.* 14, 11 (July 2021), 1950–1963. <https://doi.org/10.14778/3476249.3476254>
- [41] Yuxi Liu, Xiao Hu, Pankaj Agarwal, and Jun Yang. 2026. dsb_grasp_20k.sql. https://github.com/louisja1/bacon/blob/main/workload/dsb_grasp_20k.sql
- [42] Yuxi Liu, Xiao Hu, Pankaj Agarwal, and Jun Yang. 2026. [Full Version] BaCon: Efficient Batch Processing of Counting Queries. https://github.com/louisja1/light/sub_plan_queries/job_light_single_table_sub_query.sql

- bacon/blob/main/fullversion.pdf
- [43] Yuxi Liu, Xiao Hu, Pankaj Agarwal, and Jun Yang. 2026. Github Repository of BaCon. <https://github.com/louisja1/bacon>
 - [44] Yuxi Liu, Xiao Hu, Pankaj Agarwal, and Jun Yang. 2026. Script for generating dsb_grasp_20k.sql. https://github.com/louisja1/bacon/blob/main/workload/raw/dsb_grasp_csv_to_sql.py
 - [45] Samuel Madden, Mehul Shah, Joseph M. Hellerstein, and Vijayshankar Raman. 2002. Continuously adaptive continuous queries over streams. In *Proceedings of the 2002 ACM SIGMOD International Conference on Management of Data* (Madison, Wisconsin) (SIGMOD '02). Association for Computing Machinery, New York, NY, USA, 49–60. <https://doi.org/10.1145/564691.564698>
 - [46] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Nesime Tatbul, Mohammad Alizadeh, and Tim Kraska. 2021. Bao: Making Learned Query Optimization Practical. In *Proceedings of the 2021 International Conference on Management of Data* (Virtual Event, China) (SIGMOD '21). Association for Computing Machinery, New York, NY, USA, 1275–1288. <https://doi.org/10.1145/3448016.3452838>
 - [47] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Chi Zhang, Mohammad Alizadeh, Tim Kraska, Olga Papaemmanouil, and Nesime Tatbul. 2019. Neo: a learned query optimizer. *Proc. VLDB Endow.* 12, 11 (July 2019), 1705–1718. <https://doi.org/10.14778/3342263.3342644>
 - [48] Amine Mhedhbi and Semih Salihoglu. 2019. Optimizing subgraph queries by combining binary and worst-case optimal joins. *Proc. VLDB Endow.* 12, 11 (July 2019), 1692–1704. <https://doi.org/10.14778/3342263.3342643>
 - [49] Microsoft. [n.d.]. Microsoft SQL Server: Common Language Runtime (CLR) Integration. <https://learn.microsoft.com/en-us/sql/relational-databases/clr-integration/common-language-runtime-integration-overview?view=sql-server-ver17>
 - [50] Magnus Müller, Lucas Woltmann, and Wolfgang Lehner. 2023. Enhanced Featureization of Queries with Mixed Combinations of Predicates for ML-based Cardinality Estimation. In *Proceedings 26th International Conference on Extending Database Technology, EDBT 2023, Ioannina, Greece, March 28-31, 2023*, Julia Stoyanovich, Jens Teubner, Nikos Mamoulis, Evaggelia Pitoura, Jan Mühlig, Katja Hose, Sourav S. Bhowmick, and Matteo Lissandrini (Eds.). OpenProceedings.org, 273–284. <https://doi.org/10.48786/EDBT.2023.22>
 - [51] Parimarjan Negi, Ryan Marcus, Andreas Kipf, Hongzi Mao, Nesime Tatbul, Tim Kraska, and Mohammad Alizadeh. 2021. Flow-loss: learning cardinality estimates that matter. *Proc. VLDB Endow.* 14, 11 (July 2021), 2019–2032. <https://doi.org/10.14778/3476249.3476259>
 - [52] Parimarjan Negi, Ziniu Wu, Andreas Kipf, Nesime Tatbul, Ryan Marcus, Sam Madden, Tim Kraska, and Mohammad Alizadeh. 2023. Robust Query Driven Cardinality Estimation under Changing Workloads. *Proc. VLDB Endow.* 16, 6 (Feb. 2023), 1520–1533. <https://doi.org/10.14778/3583140.3583164>
 - [53] Hung Q. Ngo. 2018. Worst-Case Optimal Join Algorithms: Techniques, Results, and Open Problems. In *Proceedings of the 37th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems* (Houston, TX, USA) (PODS '18). Association for Computing Machinery, New York, NY, USA, 111–124. <https://doi.org/10.1145/3196959.3196990>
 - [54] Hung Q. Ngo, Ely Porat, Christopher Ré, and Atri Rudra. 2012. Worst-case Optimal Join Algorithms. arXiv:1203.1952 [cs.DB] <https://arxiv.org/abs/1203.1952>
 - [55] Hung Q. Ngo, Christopher Ré, and Atri Rudra. 2014. Skew strikes back: new developments in the theory of join algorithms. *SIGMOD Rec.* 42, 4 (Feb. 2014), 5–16. <https://doi.org/10.1145/2590989.2590991>
 - [56] Dan Olteanu and Jakub Zavodny. 2012. Factorised representations of query results: size bounds and readability. In *15th International Conference on Database Theory, ICDT '12, Berlin, Germany, March 26-29, 2012*, Alin Deutsch (Ed.). ACM, 285–298. <https://doi.org/10.1145/2274576.2274607>
 - [57] Oracle. [n.d.]. Oracle Database: External Procedures. <https://docs.oracle.com/en/database/oracle/oracle-database/19/ntqrf/external-procedures-overview.html>
 - [58] Y. Park, S. Zhong, and B. Mozafari. 2020. Quicksel: Quick selectivity learning with mixture models. In *Proc. 39th ACM SIGMOD Int. Conf. Management Data.*, 1017–1033.
 - [59] Mark Raasveldt. 2022. DuckDB - A Modern Modular and Extensible Database System. In *CDMS@VLDB*. <https://api.semanticscholar.org/CorpusID:252384081>
 - [60] Silvan Reiner and Michael Grossniklaus. 2023. Sample-Efficient Cardinality Estimation Using Geometric Deep Learning. *Proc. VLDB Endow.* 17, 4 (Dec. 2023), 740–752. <https://doi.org/10.14778/3636218.3636229>
 - [61] Wolfgang Scheufele and Guido Moerkotte. 1997. On the complexity of generating optimal plans with cross products. In *Proceedings of the Sixteenth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems*. 238–248.
 - [62] Maximilian Schleich, Dan Olteanu, Mahmoud Abo Khamis, Hung Q. Ngo, and XuanLong Nguyen. 2019. A Layered Aggregate Engine for Analytics Workloads. In *Proceedings of the 2019 International Conference on Management of Data* (Amsterdam, Netherlands) (SIGMOD '19). Association for Computing Machinery, New York, NY, USA, 1642–1659. <https://doi.org/10.1145/3299869.3324961>
 - [63] scikit-learn developers. 2007-2025. scikit-learn - RandomForestClassifier. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html#randomforestclassifier>
 - [64] Patricia G. Selinger, Morton M. Astrahan, Donald D. Chamberlin, Raymond A. Lorie, and Thomas G. Price. 1979. Access Path Selection in a Relational Database Management System. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data, Boston, Massachusetts, USA, May 30 - June 1*, Philip A. Bernstein (Ed.). ACM, 23–34. <https://doi.org/10.1145/582095.582099>
 - [65] Timos K. Sellis. 1988. Multiple-query optimization. *ACM Trans. Database Syst.* 13, 1 (March 1988), 23–52. <https://doi.org/10.1145/42201.42203>
 - [66] The Psycopg Team. 2001-2021. Psycopg - Client-side Cursors. <https://www.psycopg.org/docs/cursor.html>
 - [67] The Psycopg Team. 2001-2021. Psycopg - PostgreSQL database adapter for Python. <https://www.psycopg.org/docs/#>
 - [68] The Psycopg Team. 2001-2021. Psycopg - Server-side Cursors. <https://www.psycopg.org/docs/usage.html#server-side-cursors>
 - [69] Todd L. Veldhuizen. 2013. Leapfrog Triejoin: a worst-case optimal join algorithm. arXiv:1210.0481 [cs.DB] <https://arxiv.org/abs/1210.0481>
 - [70] Yisu Remo Wang, Max Willsey, and Dan Suciu. 2023. Free Join: Unifying Worst-Case Optimal and Traditional Joins. *Proc. ACM Manag. Data* 1, 2, Article 150 (June 2023), 23 pages. <https://doi.org/10.1145/3589295>
 - [71] Peizhi Wu and Gao Cong. 2021. A unified deep model of learning from both data and queries for cardinality estimation. In *Proceedings of the 2021 International Conference on Management of Data*. 2009–2022.
 - [72] Peizhi Wu and Zachary G. Ives. 2024. Modeling Shifting Workloads for Learned Database Systems. *Proc. ACM Manag. Data* 2, 1, Article 38 (March 2024), 27 pages. <https://doi.org/10.1145/3639293>
 - [73] Peizhi Wu, Rong Kang, Tieying Zhang, Jianjun Chen, Ryan Marcus, and Zachary G. Ives. 2025. Data-Agnostic Cardinality Learning from Imperfect Workloads. *Proc. VLDB Endow.* 18, 8 (April 2025), 2519–2532. <https://doi.org/10.14778/3742728.3742745>
 - [74] Peizhi Wu, Rong Kang, Tieying Zhang, Jianjun Chen, Ryan Marcus, and Zachary G. Ives. 2025. Original query workload of GRASP. <https://github.com/shoupzww/GRASP/blob/master/queries/dsb.csv>
 - [75] Peizhi Wu, Haoshu Xu, Ryan Marcus, and Zachary G. Ives. 2025. A Practical Theory of Generalization in Selectivity Learning. *Proc. VLDB Endow.* 18, 6 (Feb. 2025), 1811–1824. <https://doi.org/10.14778/3725688.3725708>
 - [76] Weipeng P. Yan and Per-Ake Larson. 1995. Eager Aggregation and Lazy Aggregation. In *Proceedings of the 21th International Conference on Very Large Data Bases (VLDB '95)*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 345–357.
 - [77] Zongheng Yang, Amog Kamsetty, Sifei Luan, Eric Liang, Yan Duan, Xi Chen, and Ion Stoica. 2020. NeuroCard: one cardinality estimator for all tables. *Proc. VLDB Endow.* 14, 1 (Sept. 2020), 61–73. <https://doi.org/10.14778/3421424.3421432>
 - [78] Zongheng Yang, Eric Liang, Amog Kamsetty, Chenggang Wu, Yan Duan, Xi Chen, Pieter Abbeel, Joseph M. Hellerstein, Sanjay Krishnan, and Ion Stoica. 2019. Deep unsupervised cardinality estimation. *Proc. VLDB Endow.* 13, 3 (Nov. 2019), 279–292. <https://doi.org/10.14778/3368289.3368294>
 - [79] Mihalīs Yannakakis. 1981. Algorithms for acyclic database schemes. In *Proceedings of the Seventh International Conference on Very Large Data Bases - Volume 7* (Cannes, France) (VLDB '81). VLDB Endowment, 82–94.
 - [80] Haozhe Zhang, Christoph Mayer, Mahmoud Abo Khamis, Dan Olteanu, and Dan Suciu. 2025. LpBound: Pessimistic Cardinality Estimation Using ℓ_p -Norms of Degree Sequences. *Proc. ACM Manag. Data* 3, 3 (2025), 184:1–184:27. <https://doi.org/10.1145/3725321>
 - [81] Jingren Zhou, Per-Ake Larson, Johann-Christoph Freytag, and Wolfgang Lehner. 2007. Efficient exploitation of similar subexpressions for query processing. In *Proceedings of the 2007 ACM SIGMOD International Conference on Management of Data* (Beijing, China) (SIGMOD '07). Association for Computing Machinery, New York, NY, USA, 533–544. <https://doi.org/10.1145/1247480.1247540>
 - [82] Rong Zhu, Liangui Weng, Bolin Ding, and Jingren Zhou. 2024. Learned Query Optimizer: What is New and What is Next. In *Companion of the 2024 International Conference on Management of Data* (Santiago AA, Chile) (SIGMOD '24). Association for Computing Machinery, New York, NY, USA, 561–569. <https://doi.org/10.1145/3626246.3654692>

A NOTATION TABLE

Table 6: Table of Notations.

Notation	Description
$\mathcal{D}$	Database
$\mathcal{R}$	A (sub)set of tables in $\mathcal{D}$
R	A table in $\mathcal{D}$
$\text{Attrs}[R]$	Set of attributes of table R
$\mathbf{Q}$	Set of all counting queries to evaluate over $\mathcal{D}$
Q	A counting query
Tables_Q	Set of tables referenced by Q
$\text{Preds}_Q^{\bowtie}$	Q 's join predicates
Preds_Q^σ	Q 's selection predicates
$\text{Preds}_Q^\sigma(A)$	query range for selection attribute A
$\text{Attrs}_Q^{\bowtie}$	Set of attributes referenced by Q 's join predicates
$\text{Attrs}_Q^{\bowtie}[\mathcal{R} \mathcal{R}']$	Subset of attributes of $\mathcal{R}$ used by Q to join with $\mathcal{R}'$, where $\mathcal{R}$ and $\mathcal{R}'$ are a pair of disjoint subsets of tables in Tables_Q
Attrs_Q^σ	Set of attributes referenced by Q 's selection predicates
$\text{Attrs}_Q^\sigma[\mathcal{R}]$	Subset of attributes of $\mathcal{R}$ referenced by Q 's selection predicates, where $\mathcal{R}$ is a subset of tables in Tables_Q
$\mathfrak{J} = \langle \text{Tables}_{\mathfrak{J}}, \text{Preds}_{\mathfrak{J}}^{\bowtie} \rangle$	A join pattern characterized by a set tables $\text{Tables}_{\mathfrak{J}}$ joined by predicates $\text{Preds}_{\mathfrak{J}}^{\bowtie}$ (with no selection predicates)
$\mathbf{Q}[\mathfrak{J}]$	Subset of queries in $\mathbf{Q}$ with join pattern $\mathfrak{J}$
$\text{Attrs}_{\mathfrak{J}}^\sigma$	Set of attributes involved in selection predicates in all queries of $\mathbf{Q}[\mathfrak{J}]$
$\text{Preds}_{\mathfrak{J}}^\sigma(A)$	Set of query ranges associated with selection attribute A in all queries of $\mathbf{Q}[\mathfrak{J}]$
$\mathfrak{B}_{\mathfrak{J}}$	The collection of all quantization scales for queries in $\mathbf{Q}_{\mathfrak{J}}$, one for each selection attribute in $\text{Attrs}_{\mathfrak{J}}^\sigma$, inducing a grid over a ...
$\mathfrak{B}_{\mathfrak{J}}[R]$	Subset of quantization scales for selection attributes in $\text{Attrs}_{\mathfrak{J}}^\sigma$ that belong to table R
$\mathbf{b}_A \in \mathfrak{B}_{\mathfrak{J}}$	Quantization scale for attribute $A \in \text{Attrs}_{\mathfrak{J}}^\sigma$
$\mathbf{b}_A(x)$	Serial integer id of the bucket containing x (or 0 if x lies outside all buckets), where x is a value from A 's domain
$\vec{b}$	A grid coordinate, with one integer component per selection attribute
$\mathfrak{M}$	A count map
$\mathfrak{M}[\vec{b}]$	The count of tuples falling within the grid cell with coordinate $\vec{b}$
β	Parameter used by optimized BACONRECURSE to batch PROCESSTABLE calls (Section 4.2)

B MORE DETAILS OF BASIC BACON

We present the detailed construction algorithm of $\mathbf{b}_A$ for a selection attribute A in Appendix B.1.

B.1 Construction of Buckets

Continuing from Section 3.2, we extract from $\mathbf{Q}[\mathfrak{J}]$ the set $\Delta = \{\text{Preds}_Q^\sigma(A) \mid \exists Q \in \mathbf{Q}[\mathfrak{J}] : A \in \text{Preds}_Q^\sigma\}$ of predicate ranges associated with A . We transform each range G specified by $\text{Preds}_Q^\sigma(A)$ to

a left-closed, right-open interval. Concretely, a left-open boundary (l) is converted to a left-closed boundary $\text{succ}(l)$, and a right-closed boundary (r) is converted to a right-open boundary $\text{succ}(r)$, where $\text{succ}(\cdot)$ advances the value by one unit at the attribute's resolution⁷. Next, we sort the boundaries in ascending order and remove duplicates. The i -th boundary and the $(i+1)$ -th one (if exists), where there are more left boundaries than right ones among the first i boundaries, form a left-closed-right-open interval in the domain of A that is of interest to at least one query. We refer to each of these intervals as a bucket, and all these buckets form $\mathbf{b}_A$. As detailed in Section 3.2, a value v is quantized into an integer, i.e., the id of the bucket containing v , or 0 if v lies outside all of the buckets in $\mathbf{b}_A$. Note that if a query in $\mathfrak{J}$ has no selection predicates on A , it still satisfies Lemma 3.2 by setting $i_1 = 0$ and $i_2 = |\mathbf{b}_A|$.

We present an example of quantization in Example B.1.

EXAMPLE B.1. *Continuing from Listing 1, we have*

$$\mathbf{b}_{\text{c.Score}} = \{[0, 1)\}$$

,

$$\mathbf{b}_{\text{ph.PostHistoryTypeId}} = \{[1, 2)\}$$

, and

$$\mathbf{b}_{\text{ph.CreationDate}} = \{[2010-09-14\ 11:59:07, \infty)\}$$

.

By Lemma 3.2, the integer range associated with Q_3 is $[1, 1]$ for c.Score , $[1, 1]$ for $\text{ph.PostHistoryTypeId}$, and $[0, 1]$ for ph.CreationDate . Similarly, the integer range associated with Q_4 is $[0, 1]$ for c.Score , $[1, 1]$ for $\text{ph.PostHistoryTypeId}$, and $[1, 1]$ for ph.CreationDate .

⁷For example, if α is an integer, $\text{next}(\alpha) = \alpha + 1$; if α is a timestamp without time zone, $\text{next}(\alpha) = \alpha + 1$ microsecond; and so on.

Listing 4: SQL code for β -PROCESSTABLE($R, U, \mathcal{A}^{\text{in} \& \text{out}}$).

Input: Set U of mappings, with each binds attributes $J_1^{\text{in}}, J_2^{\text{in}}, \dots$ to specific values, and $\mathcal{A}^{\text{in} \& \text{out}} = \{J_1^{\text{in}}, J_2^{\text{in}}, \dots, J_1^{\text{out}}, J_2^{\text{out}}, \dots\}$ specifies the set of attributes for partitioning result entries. The quantization scales are $\mathfrak{B}_3[R] = \{b_{S_1}, b_{S_2}, \dots\}$.

Output: Result entries have the form $\langle v, \tilde{b}, c \rangle$, sorted by v , where v is a mapping from $\mathcal{A}^{\text{in} \& \text{out}}$ to values, $\tilde{b}$ is a grid coordinate for $\mathfrak{B}_3[R]$, and c is the associated count.

```
SELECT J1in, J2in, ..., J1out, J2out, ..., B1, B2, ..., COUNT(*)
FROM (
  SELECT J1in, J2in, ..., J1out, J2out, ...,
    quantize(S1, bS1) AS B1, quantize(S2, bS2) AS B2, ...
  FROM R
  WHERE J1in BETWEEN min(U(J1in)) AND max(U(J1in))
    J2in BETWEEN min(U(J2in)) AND max(U(J2in)) AND ...
) AS TMP
GROUP BY J1in, J2in, ..., J1out, J2out, ..., B1, B2, ...
ORDER BY J1in, J2in, ..., J1out, J2out, ...;
```

C MORE DETAILS OF BACON IMPLEMENTATION

We present the details of β -PROCESSTABLE and β -BACONRECURSE in Appendix C.1.

C.1 Details of β -PROCESSTABLE and β -BACONRECURSE

In β -PROCESSTABLE (Listing 4), the second and third arguments differ from those of PROCESSTABLE. Specifically, β -PROCESSTABLE takes a set U of bindings rather than a single binding u , and it is passed both $J_1^{\text{in}}, J_2^{\text{in}}, \dots$ and $J_1^{\text{out}}, J_2^{\text{out}}, \dots$ to enable partition of result entries. Moreover, in the inner block, $J_1^{\text{in}}, J_2^{\text{in}}, \dots$ are filtered using safe but potentially imprecise range bounds. Concretely, $\min U(J_i^{\text{in}})$ (resp. $\max U(J_i^{\text{in}})$) denotes the minimum (resp. maximum) value in the set $\{u(J_i^{\text{in}}) \mid u \in U\}$. Finally, β -PROCESSTABLE also SELECTs $J_1^{\text{in}}, J_2^{\text{in}}, \dots$ and partitions (and orders) the result entries by these attributes, allowing the receiver, β -BACONRECURSE, to discard tuples that do not correspond to any binding in U .

We propose β -BACONRECURSE (Algorithm 3) by incorporating batching into BACONRECURSE. Instead of producing a single count map, β -BACONRECURSE outputs a dictionary that maps each binding $u \in U$ to a corresponding count map, enabling batch processing of β consecutive bindings. Lines 3-29 enumerate subsequences of result entries. Unlike BACONRECURSE, these subsequences are not processed immediately; computation is deferred until either β subsequences have been accumulated or the end of the returned entries is reached (Line 9). As discussed earlier, returned tuples may not belong to U . Therefore, a post-check filters tuples by testing whether v 's projection onto $J_1^{\text{in}}, J_2^{\text{in}}, \dots$ is contained in U (Lines 4-7). During batch processing (Lines 9-28), we first collect the set U_i of distinct bindings on the join attributes between the child table R_i and R (Lines 12-16). We then recursively invoke β -BACONRECURSE for each pair (R_i, U_i) . Finally, Lines 19-26 update the dictionary $\mathbf{M}$ by aggregating the contribution of each subsequence. The procedure for merging count maps largely follows BACONRECURSE, with two key differences: R_i 's count map is indexed by u_i and obtained from the returned dictionary; and the merged count maps are accumulated into the dictionary entry indexed by ω^{in} . Note that ω^{in} (and

similarly v^{in} in Line 4) is empty for the root table, which has no J^{in} attributes.

Algorithm 3 β -BACONRECURSE(R, U)

Input: Set U of mappings, with each binds a subset of R 's attributes to specific values. Implicitly, the function also has access to $\mathcal{D}$, $\mathfrak{J}$ and its plan tree, and the quantization scales $\mathfrak{B}_3$.

Output: Dictionary $\mathbf{M}$ maps each binding $u \in U$ to a count map $\mathfrak{M}$ over subtree(R); formally, $\mathbf{M} := \{u \mapsto \mathfrak{M} \mid u \in U\}$. Each $\mathfrak{M}$ (or equivalently $\mathbf{M}[u]$) is complete w.r.t. result tuples of $\mathcal{Q}[\mathfrak{J}]$ restricted to subtree(R) and consistent with u .

```
1:  $\mathbf{M} \leftarrow \{u \mapsto \text{an empty count map} \mid \forall u \in U\}$ ;
2:  $n_{\text{subsequence}} \leftarrow 0$ ;  $\triangleright$  counter for the number of subsequences to be processed
3: for each  $S[v]$  of entries of the form  $\langle v, \cdot, \cdot \rangle$  with the same  $v$ , returned by  $\beta$ -PROCESSTABLE( $R, U, \text{Attrs}_{\mathfrak{J}}^{\text{in}}(\text{parent}(R) \mid R) \cup \text{Attrs}_{\mathfrak{J}}^{\text{in}}(R \mid \text{children}(R))$ ) do
4:    $v^{\text{in}} \leftarrow \langle v[J_1^{\text{in}}], \tilde{J}_2^{\text{in}}, \dots \rangle$ ;  $v^{\text{out}} \leftarrow \langle v[J_1^{\text{out}}], \tilde{J}_2^{\text{out}}, \dots \rangle$ ;
5:   if  $v^{\text{in}} \notin U$  then  $\triangleright$  the false positive case mentioned in Section 4.2: join attribute values lie in  $[\min(U), \max(U)]$  but are not contained in  $U$ 
6:     continue;
7:   end if
8:    $n_{\text{subsequence}} \leftarrow n_{\text{subsequence}} + 1$ ;
9:   if  $n_{\text{subsequence}} = \beta$  or  $S[v]$  is the last subsequence returned by  $\beta$ -PROCESSTABLE then  $\triangleright$  batch processing of  $\beta$  subsequences; equivalently, batch processing a set  $V$  of consecutive values of  $v$ 
10:    for each table  $R_i \in \text{children}(R)$  do
11:       $U_i \leftarrow \emptyset$ ;
12:      for each subsequence  $S[\omega]$  in the batch do  $\triangleright$  each subsequence in the batch ended with  $S[v]$ 
13:         $\omega^{\text{out}} \leftarrow \langle \omega[J_1^{\text{out}}], \omega[J_2^{\text{out}}], \dots \rangle$ ;
14:         $u_i \leftarrow \left\{ A' \mapsto \omega^{\text{out}}(A) \mid \begin{array}{l} A \in \text{Attrs}_{\mathfrak{J}}^{\text{in}}(R \mid R_i) \\ \wedge A' \in \text{Attrs}_{\mathfrak{J}}^{\text{in}}(R_i \mid R) \\ \wedge \text{Preds}_{\mathfrak{J}}^{\text{in}} \Rightarrow (A = A') \end{array} \right\}$ ;
15:         $U_i \leftarrow U_i \cup u_i$ ;
16:      end for
17:       $\mathbf{M}_i \leftarrow \beta\text{-BACONRECURSE}(R_i, U_i)$ ;
18:    end for
19:    for each subsequence  $S[\omega]$  in the batch do
20:       $\omega^{\text{in}} \leftarrow \langle \omega[J_1^{\text{in}}], \omega[J_2^{\text{in}}], \dots \rangle$ ;  $\omega^{\text{out}} \leftarrow \langle \omega[J_1^{\text{out}}], \omega[J_2^{\text{out}}], \dots \rangle$ ;
21:       $\mathfrak{M}_0 \leftarrow \{b \mapsto c \mid \langle \omega, \tilde{b}, c \rangle \in S[\omega]\}$ ;
22:      for each table  $R_i \in \text{children}(R)$  do
23:         $\mathfrak{M}_0 \leftarrow \mathfrak{M}_0 \otimes \mathbf{M}_i[u_i]$ ;  $\triangleright u_i$  as defined in Line 14
24:      end for
25:       $\mathbf{M}[\omega^{\text{in}}] \leftarrow \mathbf{M}[\omega^{\text{in}}] \oplus \mathfrak{M}_0$ ;
26:    end for
27:     $n_{\text{subsequence}} \leftarrow 0$ ;
28:  end if
29: end for
30: return  $\mathbf{M}$ ;
```

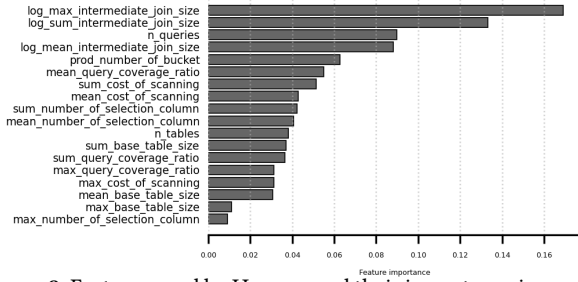


Figure 8: Features used by HYBRID, and their importance in a model trained as described in Section 5.4.

C.2 Choosing among BACon/INDPROC/PostFILT

We introduce an approach called *HYBRID* for picking the appropriate method to use among the three for a given query pattern. As we will see in Section 5.4, BACon performs well across our target workloads, so HYBRID only serves to validate the robustness of BACon. Nonetheless, we briefly describe HYBRID here for completeness.

We train HYBRID from sample workloads as a classifier that predicts, given a join pattern, one of three classes {0, 1, 2} representing BACon, INDPROC, and PostFILT respectively. We use a lightweight model based on `RandomForestClassifier` [63] over a suite of features — the most important of which are illustrated in Figure 8. Features with prefixes `sum_`, `max_`, `mean_`, and `prod_` aggregate a collection of values: for example, `prod_number_of_bucket` is the product of the number of buckets across all quantization scales for a join pattern; `mean_query_coverage_ratio` is the average, taken per query, of the ratio between the number of grid cells queried and `prod_number_of_bucket`. Features containing `intermediate_join_size` are AGM bounds [6, 16] (i.e., overestimation) of intermediate result sizes of join subplans chosen by PostgreSQL for joining all tables in the join pattern. Features with `cost_of_scanning` denote PostgreSQL’s estimated cost for enumerating a table ordered by join attributes that connects it to its descendant tables (if any), reflecting a key component of BACon. In general, we include as features per-join-pattern statistics that are inexpensive to collect at runtime. However, we exclude more advanced selectivity estimates provided by PostgreSQL, because we have found them to be unreliable and systematically underestimating the costs of baselines.

To ensure a lightweight model and fast inference, we train our model with `train_val_split=0.25`, `n_estimators=100`, `min_samples_leaf=2`, and `class_weight={“0”: 1, “1”: 2, “2”: 2}`. The higher weights for classes 1 and 2 penalize mistakenly picking baseline approaches, because such mistakes cost much higher. To obtain class labels for training data, we measure and compare the running time for all three methods. We deem a baseline method *safe* if its running time is either (i) at least 1.5× faster than BACon, or (ii) no less than 10 seconds faster than BACon. We assign class 1 only when INDPROC is safe and PostFILT is either unsafe or slower; class 2 is assigned analogously; all remaining cases are labeled as class 0. For inference, rather than selecting the class with the highest posterior probability, we require the predicted probability of choosing INDPROC or PostFILT to exceed a precision threshold (0.9 by default), calibrated to guarantee minimum precision on held-out validation data. If no probability threshold can guarantee the required precision, the one with highest precision is selected. This robust thresholding strategy

deliberately trades recall for safety, selecting baselines only under high confidence.

D MORE DETAILS OF EXPERIMENTS

We present the results of materialized-view-based postfiltering in Appendix D.1, the results of multi-threaded execution in Appendix D.2, and the results of varying β in Appendix D.3. We present the per-join-pattern results of both the remaining real workloads and scalability workloads in Appendix D.4. Additionally, the most detailed result of BACON and POSTFILT can be found in the log files of the form `results/*.print` in our repository [43].

D.1 Experiments of Materialized-View-Based Postfiltering

An alternative implementation of POSTFILT materializes the unfiltered join for each join pattern and performs aggregations over it. However, this approach is impractical for join patterns with very large unfiltered join sizes, plus this information is not free, due to prohibitive storage overhead. Therefore, we evaluate the materialized-view-based variant only on DSB’s `dsb-grasp-20k` and JOB’s `synthetic`, where the materialized view cardinalities remain manageable and POSTFILT outperforms INDPROC (as shown in Table 2). The result is presented in Table 7.

Table 7: End-to-end running times (in seconds, rounded) of the materialized-view-based algorithm, compared to POSTFILT.

Query Workload	View Cardinalities	POSTFILT	Materialized-view-based Postfiltering
dsb-grasp-20k	$< 6 \cdot 10^6$	4,211	4,356
synthetic	$< 5 \cdot 10^8$	11,329+	12,264+

D.2 Experiments of Multi-threaded Execution

We reports results for INDPROC, POSTFILT, and BACON under multi-threaded execution. For the baselines, INDPROC and POSTFILT, we enable PostgreSQL’s parallel execution by setting `max_parallel_workers` to 8 and `max_parallel_workers_per_gather` to 4. Since all their computation is performed within the database server, they can fully exploit this parallelism. In contrast, without an elaborate parallelization design, BACON uses 4 processes, each handling a subset of join patterns assigned in a round-robin manner, while keeping the PostgreSQL server in single-threaded mode. The results are presented in Table 8. While we adopt this simple, join-pattern-level parallelization, more sophisticated strategies for BACON are possible (e.g., parallel merging of count maps from descendants or cost-based partitioning of join patterns), while we leave as future work.

D.3 Experiments of Various β

We report results for $\beta = 5,000$, $50,000$ and $500,000$ respectively, with $\beta = 50,000$ as the default setting used throughout Section 5. The corresponding end-to-end running times are shown in Table 9. Overall, increasing β reduces the end-to-end running time, at the cost of higher space overhead (e.g., larger M_i in Line 17 of Algorithm 3). As indicated by the last two columns of Table 9, the primary source of performance differences lie in the number of SQLs issued to the underlying database and the time spent fetching their results.

Table 8: Multi-threaded mode: end-to-end running times (in seconds, rounded) and relative speedups. +: lower-bound values, as INDPROC is capped at 900s per query and POSTFILT at 3,600s per join pattern.

Query Workload	INDPROC (parallel)	POSTFILT (parallel)	BACON (parallel)	Speedup vs. INDPROC
synthetic	5,271	2,565	531	$\uparrow 9.92 \times$
scale	11,003+	14,715+	1,097	$\uparrow 10.03+ \times$
job-light	540	8,046	273	$\uparrow 1.98+ \times$
job-light-single	66	3	11	$\uparrow 5.83 \times$
job-light-join	2,595	11,198+	510	$\uparrow 5.09 \times$
stats-ceb	3,390+	43,661+	19	$\uparrow 180.79+ \times$
stats-ceb-single	17	2	2	$\uparrow 8.48 \times$
stats-ceb-join	32,384+	51,120+	54	$\uparrow 598.48+ \times$
dsb-grasp-20k	6,241	913	359	$\uparrow 17.37 \times$

With larger β s, BACON issues fewer SQLs and incurs lower result-fetching overhead, resulting in more savings on the end-to-end running times.

Table 9: job-light: end-to-end running times (in seconds, rounded), together with the total number of SQLs executed on the underlying database and the total time spent fetching their results, for different values of β . The default configuration in Section 5 ($\beta = 50,000$) is shown in bold.

β	E2E running time	Total # of SQLs	Total result-fetching overhead
5,000	922	19,427	639
50,000	728	1,974	497
500,000	584	243	288

D.4 More Per-Join-Pattern Results

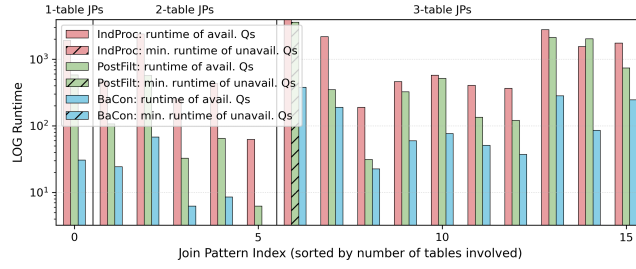


Figure 9: Log-scale running time per join pattern in synthetic.

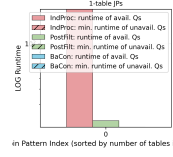


Figure 10: Log-scale running time per join pattern in job_light_single.

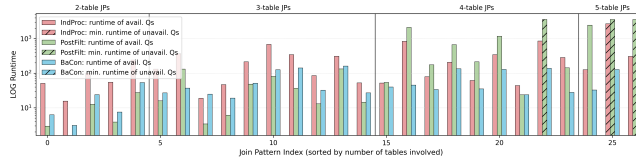


Figure 11: Log-scale running time per join pattern in job_light_join.

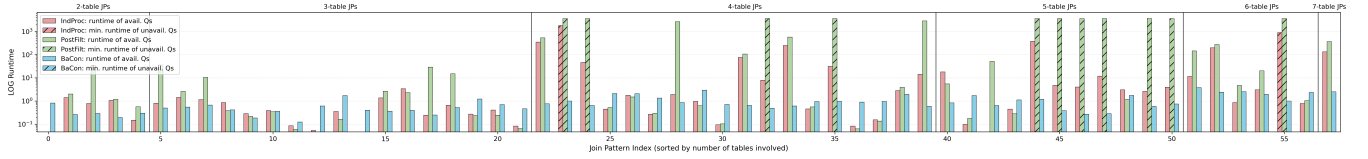


Figure 12: Log-scale running time per join pattern in stats_ceb.

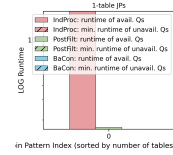


Figure 13: Log-scale running time per join pattern in stats_ceb_single.

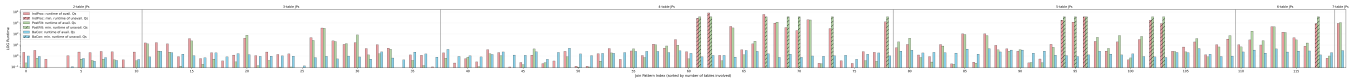


Figure 14: Log-scale running time per join pattern in stats_ceb_join.

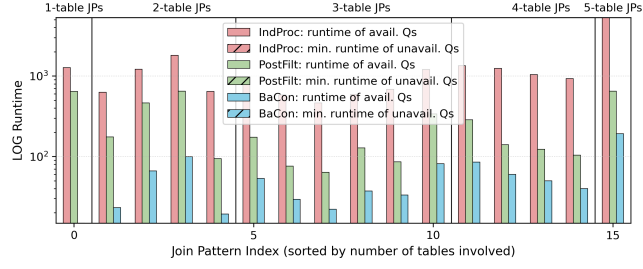


Figure 15: Log-scale running time per join pattern in dsb_grasp_20k.

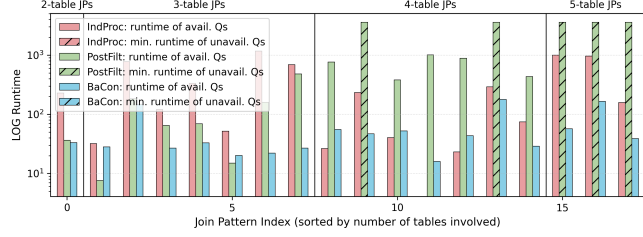


Figure 16: Log-scale running time per join pattern in job_light_1k.

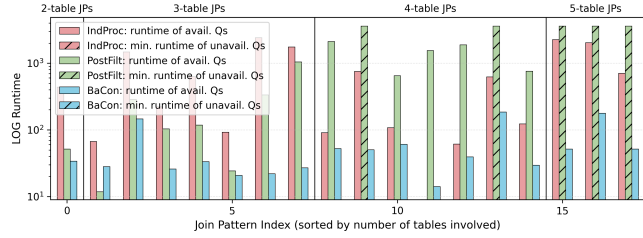


Figure 17: Log-scale running time per join pattern in job_light_2k.

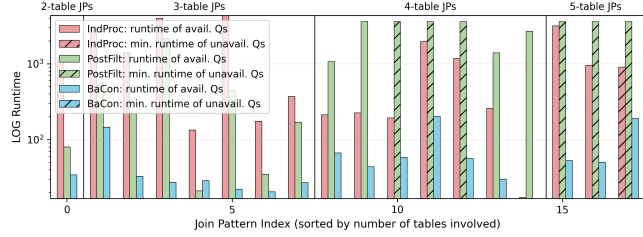


Figure 18: Log-scale running time per join pattern in job_light_4k.

E MORE DETAILS OF RELATED WORK

Select-Join-Aggregate Query Processing. Counting the join results (with selection predicate on the base tables) is a special class of select-join-aggregate queries (intuitively, applying an aggregate function on top of the select-join queries) with the output size as 1. Processing such queries usually first push down the selection predicate to the base table, and ends up with processing join-count queries, which is the focus below.

The previous works have achieved two flavors of results: worst-case optimal and output-sensitive. Namely, worst-case optimal algorithms work only on pathological instances with huge outputs, which are rare in practice. In contrast, output-sensitive algorithms express the runtime as a function of the input size and output size, which are more practically meaningful, especially for queries where the aggregation (such as count) may significantly reduce the output size. The classical Yannakakis algorithm [79] can compute *free-connex* join-count queries in $O(N)$ time, where N is the total number of tuples in the database. Very recently, Hu [26] exploits the hybrid strategy of Yannakakis algorithms, which can compute acyclic join-count queries in $O(N \cdot \text{OUT}^{1 - \frac{1}{\text{fnfhtw}}} + \text{OUT})$ time, where

OUT is the size of the projection of the join results onto the counting columns, and *fnfhtw* is the free-connex fractional hypertree width of the query. For general join-count queries, the state-of-the-art approach is to convert the query into a free-connex one using the tree decomposition technique and the worst-case optimal join algorithm [53–55, 69], and then run the hybrid Yannakakis algorithm on the tree decomposition.

Our work is also inspired by some ideas in these works. The instantiations of worst-case optimal join algorithms [2, 14, 48, 70] motivate BACON to decompose computation via child recursions and combine multiple coordinate-to-count maps during backtracking. Cost-based optimizers in these systems, such as Free Join’s [70], together with the AGM bound [6, 16], further inform our initial exploration of cost-based hybrid mechanism. However, achieving more accurate cost estimates for BACON may conversely require more accurate (pessimistic) cardinality estimation at a higher cost, such as LpBound [80], and/or specialized data storages and execution engines, such as Free Join’s COLT [70] and its integration to DuckDB [59].

Aggregation Push-down. Early works explore aggregation push-down in query plans to reduce intermediate result sizes. Eager aggregation and lazy aggregation [76] introduce rules for early or deferred partial aggregation based on algebraic properties, while Chaudhuri and Shim [9] integrate GROUP BY into cost-based optimization; both focus on intra-query optimization via query rewriting. More recently, GuAo [33] identifies aggregate queries that can be evaluated without materializing any joins and introduces a corresponding physical operator for SparkSQL [5]. GuAo targets single-query execution and assumes selection push-down at table scans. For the case without GROUP BY, its propagation of frequencies grouped by join attributes resembles BACON’s bucketization when reduced to a single-query join pattern. Unlike BACON, however, GuAo merges frequencies via standard binary joins in SparkSQL rather than supporting multi-way combination during count-map merging.

Factorizations. Factorized databases (FDBs) propose the factorized representations of query results [8] to eliminate tuple-level redundancy and boost the relational processing performance [56], supporting aggregations beyond COUNT [7]. These works focus on intra-query redundancy in query output and are different but complementary to our approach. Moreover, they assume storage layout different from traditional relational databases, like PostgreSQL.

Optimized Batch Processing of Aggregate Queries. LMFAO [62] is a framework proposed to efficiently compute a batch of group-by aggregates over shared joins without fully materializing intermediate results. It decomposes queries into views, groups them according to a join tree, and evaluates each view group using a multi-output plan. Similar to POSTFILT, LMFAO supports selection predicates by encoding them as conditional expressions inside aggregate functions. In contrast to BACON, LMFAO does not analyze or pre-process overlaps among cross-query predicates (e.g., via bucket construction), but instead focuses on sharing computation at the level of join tree traversal and attribute-ordered evaluation, reusing identical conditional expressions and partial products within a view group. Moreover, LMFAO is implemented as a standalone engine, rather than being integrated into a traditional DBMS.

Multi-Query Optimization & Shared Workload Optimization. Multi-query optimization (MQO) [65] identifies shared or similar subexpressions across queries and performs inter-query optimization by executing the common subexpressions jointly or by rewriting query plans with a boarder subquery. Along this line, Zhou et al. [81] propose a practical common subexpression manager integrated into Microsoft SQL Server and its cost-based query optimizer, demonstrating improvements on workloads of a few tens of queries. As the search space for identifying common subplans grow rapidly with the number of queries, Shared Workload Optimization (SWO) [15] optimizes an entire workload – often comprising hundreds or thousands of concurrent queries – by identifying shared operators instead. This typically requires specialized or substantially modified execution engines (e.g., shared work systems) to support shared physical operators and coordinated execution. Both lines of work target more general queries and require accurate cost estimates, which motivates future extensions of BACON toward tighter, cost-based integration with DBMSs.

Continuous Query Processing. Related works on continuous query processing [3, 45] in stream systems explore shared computations by maintaining data summaries or predicate indices over evolving streams. These approaches are designed for workloads with large

numbers of continuous queries or filters (e.g., on the order of 100k), usually exceeding the workload sizes considered in our setting, and rely on specialized structures to efficiently support updates. Nevertheless, they offer complementary insights that could inform strategies for handling workloads at different scales as future work.

Learned Cardinality Estimation. As the main motivation of BACON, query-aware CE models [13, 27, 29, 37, 39, 40, 50, 51, 58, 71] learns to predict cardinalities from a training set of query-count pairs. Training is typically performed either in a single-shot manner, where a model is (re-)trained from scratch on a collected batch of queries [13, 27, 37, 38, 40], or in multiple rounds, where the model is incrementally updated as new query-count pairs occur (e.g., via fine-tuning) [25, 39, 58]. The required input information varies across approaches. Pure data-driven CE models [77, 78] learn data distributions directly from the tables and do not require query-count pairs for training, although query workloads are still used for evaluation. Some hybrid models [29, 39] additionally require data statistics, such as samples or histograms. Beyond model design, recent work has focused on model robustness [32, 75], training efficiency [60], and adaptivity to updates and drifts [25, 37, 39, 52, 72], which further increases the demand for diverse query workloads.